

Quantum Bayesian Networks Can Speed up Reinforcement Learning in Partially Observable Environments

Gilberto Cunha^{b,c}, Alexandra Ramôa^{a,b,c,*}, André Sequeira^{a,b,c}, Michael de Oliveira^{a,c}, Luís Barbosa^{a,b,c}

^a*High-Assurance Software Laboratory (HASLab), INESC TEC, Portugal*

^b*Department of Computer Science, University of Minho, Portugal*

^c*International Iberian Nanotechnology Laboratory (INL), Portugal*

Abstract

Reinforcement learning (RL) provides a principled framework for decision-making in partially observable environments, which can be modeled as Markov decision processes and compactly represented through dynamic decision Bayesian networks. Recent advances demonstrate that inference on sparse Bayesian networks can be accelerated using quantum rejection sampling combined with amplitude amplification, leading to a computational speedup in estimating acceptance probabilities.

Building on this result, we introduce Quantum Bayesian Reinforcement Learning (QBRL), a hybrid quantum-classical look-ahead algorithm for model-based RL in partially observable environments. We present a rigorous, oracle-free time complexity analysis under fault-tolerant assumptions for the quantum device. Unlike standard treatments that assume a black-box oracle, we explicitly specify the inference process, allowing our bounds to more accurately reflect the true computational cost. We show that, for environments whose dynamics form a sparse Bayesian network, horizon-based near-optimal planning can be achieved sub-quadratically faster through quantum-enhanced belief updates. On the other hand, we show that there is no quantum speed-up for environments that are either fully observable, or characterized by Bayesian networks whose maximum in-degree is not small.

Furthermore, we present numerical experiments benchmarking QBRL against its classical counterpart on simple yet illustrative decision-making tasks. Our results offer a detailed analysis of how the quantum computational advantage translates into decision-making performance, highlighting that the magnitude of the advantage can vary significantly across different deployment settings.

Keywords: Reinforcement learning, partially observable Markov decision process, Bayesian networks, quantum algorithms, quantum amplitude amplification,

*Corresponding author: alexandra.ramoa@inl.int.

1. Introduction

Reinforcement learning (RL) is a foundational paradigm for sequential decision-making under uncertainty, enabling agents to learn optimal strategies by interacting with and adapting to dynamic environments. It formalizes decision-making as a reward optimization problem, and plays a central role in artificial intelligence (AI), especially in scenarios where explicit supervision is unavailable or partial. RL has been pivotal to some of the most significant advances in modern AI, including AlphaGo’s mastery of the game of Go Silver et al. (2016) and more recently, the outstanding performance improvement of large language models Wang et al. (2025). RL has also shown impact in domains such as finance, robotics, natural language processing, and metrology Fiderer et al. (2021); Nian et al. (2020); Bai et al. (2024). These successes highlight RL’s versatility, but also its computational demands—particularly in high-dimensional, partially observable, or data-scarce settings—where improvements in sample efficiency and computational scalability are critical.

Quantum advantages for RL have been investigated in both tabular and approximate settings Meyer et al. (2024). In the tabular regime, Dunjko et al. (2018) suggests that Quantum Reinforcement Learning (QRL) may yield polynomial to exponential improvements under various conditions. This result has been further substantiated by recent contributions: Sannia et al. (2023) employed quantum amplitude amplification to accelerate Q-value estimation in Q-learning; Cherrat et al. (2023a) introduced a QRL framework based on policy iteration, showing that quantum algorithms for policy evaluation and improvement can achieve meaningful speedups; and Ganguly et al. (2024) demonstrated that logarithmic regret is achievable in model-based episodic and average-reward Markov Decision Processes (MDPs), suggesting theoretical benefits for quantum agents in structured environments.

In the approximate setting, quantum-enhanced policies based on parameterized quantum circuits (PQCs) have also shown promise. Empirical studies indicate that PQC-based models can improve agent convergence in various scenarios Jerbi et al. (2021); Skolik et al. (2022). Jerbi et al. (2022) established that PQC-based policies can be trained quadratically faster when granted full access to quantum environments. Most recently, Xu & Aggarwal (2025) demonstrated that PQC-based model-free agents for infinite-horizon MDPs can attain a computational speedup that exceeds known classical lower bounds—underscoring the concrete benefits of fully quantum reinforcement learning approaches.

However, none of these advances explicitly addresses partially observable environments, which are significantly more complex than fully observable ones (PSPACE-complete Papadimitriou & Tsitsiklis (1987)), and thus more computationally challenging. Tackling this problem is paramount, as partial observability arises in a multitude of real-world scenarios, due to noisy sensors or otherwise limited information Muškardin et al. (2024). In practice, sensors are often inaccurate, or operate under conditions that constrain their ability to perceive the environment, such as limited visibility.

In such environments, RL problems are typically modeled as POMDPs and can be compactly encoded as dynamic decision Bayesian networks (DDNs). A central challenge lies in efficiently performing belief updates and inference over these probabilistic graphical models.

Quantum computing offers promising tools to tackle these difficulties. In particular, quantum rejection sampling combined with amplitude amplification Grover (1996); Brassard et al. (2002); Low et al. (2014a) provides a quadratic speedup in the success probability of sampling for sparse Bayesian networks. Additionally, de Oliveira & Barbosa (2021) introduced an algorithm that leverages quantum amplitude amplification to obtain provable quantum advantages in static decision-making tasks—without requiring explicit evaluation of conditional probabilities, while Gao et al. (2022) demonstrated that quantum correlations enhance the expressive power of generative models such as Bayesian networks. Their findings suggest that minimal quantum extensions may improve sample complexity and convergence of quantum reinforcement learning agents given partially observable dynamics.

Building on this result, we introduce a hybrid quantum-classical look-ahead algorithm for model-based RL in partially observable environments. We present a rigorous, oracle-free time complexity analysis under fault-tolerant assumptions for the quantum device. Unlike standard treatments that assume a black-box oracle, we explicitly specify the inference process, allowing our bounds to more accurately reflect the true computational cost. We show that, for environments whose dynamics form a sparse Bayesian network, horizon-based near-optimal planning can be achieved sub-quadratically faster through quantum-enhanced belief updates. We additionally present numerical experiments on simple yet illustrative decision-making tasks, showing how the quantum algorithm outperforms its classical counterpart for a fixed cost, or conversely reduces the runtime for the same performance. Furthermore, we demonstrate and discuss the dependence of this advantage on the practical task at hand. Our results highlight that the magnitude of the advantage can vary significantly across different deployment settings.

The quantum component of our algorithm handles probabilistic inference for Bayesian networks, which model the reinforcement learning environment. The task is to estimate the conditional distributions that are necessary for belief updates. In partially observable settings, some of these estimates rely on rejection sampling. The acceptance probabilities typically shrink exponentially with the number of variables, making this a bottleneck Low et al. (2014a). For sparse networks, quantum amplitude amplification can improve the efficiency of the rejection sampling sub-routine by boosting the acceptance probability Low et al. (2014a). This has the potential to speed-up reinforcement learning in some scenarios, as we rigorously analyze and numerically demonstrate.

In short, our contribution is to leverage a known quantum speed-up in rejection sampling for Bayesian networks and investigate how it can translate into a quantum speed-up for reinforcement learning. This includes a complexity analysis, numerical experiments, and a discussion of advantages and limitations.

We note that the term “Bayesian reinforcement learning” typically refers to algorithms where Bayesian statistics are used to address the exploration/exploitation trade-off Ghavamzadeh et al. (2015). These are not directly related to our work, where Bayesian networks are a tool to represent the environment. The relevant comparison is with the classical counterpart of our algorithm; accordingly, that is used as a reference throughout this work. We note the existence of other solutions to POMDPs Silver & Veness (2010); Moss

et al. (2023); Smith & Simmons (2012); Pineau et al. (2003); Spaan & Vlassis (2005), some of which can be more efficient. It would be interesting to study whether quantum speed-ups apply to these alternatives.

Paper structure. The rest of the paper is organized as follows. Section 2 overviews related work, and clarifies where this work stands with respect to it. Section 3 introduces the foundational concepts necessary to understand our hybrid quantum-classical algorithm, which is described in section 4. Section 5 presents the computational complexity of the classical and hybrid approaches. The methodology employed for the numerical evaluation of the algorithms is detailed in section 6, with the corresponding results and discussion presented in section 7. Lastly, section 8 highlights the main findings and outlines directions for future research.

2. Related work

This section situates the present work with respect to related research. Research at the interface of quantum computing and reinforcement learning has developed along several distinct directions; this section situates the present work with respect to these directions. For an overview, refer to Meyer et al. (2024).

One such direction studies reinforcement learning in settings where the agent has quantum access to the environment; this yields provable improvements for exploration, policy evaluation, policy iteration, and regret minimization Dunjko et al. (2018); Wang et al. (2021); Cherrat et al. (2023b); Ganguly et al. (2024); Jerbi et al. (2022). In contrast, our work concerns classical environments; quantum resources are used to accelerate a specific inference subroutine.

A second line of work, motivated by near-term devices, uses parameterized quantum circuits as function approximators inside otherwise classical learning loops Jerbi et al. (2021); Skolik et al. (2022). Such approaches investigate expressive quantum policies and hybrid training schemes, and in some cases show empirical or sample-complexity advantages. However, they do not target the bottleneck that arises from belief-state updates in model-based planning under partial observability, which is the focal point of this work. Furthermore, we rely on non-variational, fault tolerant circuits.

Another related direction is classical reinforcement learning as a tool for quantum control. Representative examples include deep-RL methods for quantum state preparation and gate control, as well as approaches for open quantum systems Bukov et al. (2018); Niu et al. (2019); Ernst et al. (2025). This direction too is conceptually different from our setting: reinforcement learning is the optimization tool used to solve a quantum-control problem, whereas here we consider the reverse scenario where quantum computation is a tool to accelerate reinforcement learning.

There have also been proposals of quantum-inspired algorithms for reinforcement learning Chen & Dong (2008); Kaldari et al. (2025); Yu et al. (2025). Unlike the present work, these are fully classical approaches; they are motivated by quantum mechanical principles, but do not effectively use quantum hardware.

Our work is most directly connected to quantum probabilistic inference for Bayesian networks. It has been shown that quantum rejection sampling combined with amplitude amplification offers a quadratic

improvement in the dependence on the evidence probability for sparse Bayesian networks Low et al. (2014a). Quantum-circuit constructions for Bayesian and dynamic Bayesian networks were subsequently developed in Borujeni et al. (2021), while Gao et al. (2022) showed that quantum correlations can enhance the expressive power of generative models related to Bayesian networks. We build on this inference perspective and ask how such an advantage propagates to sequential decision making when a partially observable environment is represented as a dynamic decision network.

Finally, our paper also relates to the vast literature on classical planning and learning in POMDPs. Many paths have been proposed for solving partially observable problems, such as point-based methods, online Monte Carlo planning methods, and methods leveraging learned approximations Pineau et al. (2003); Spaan & Vlassis (2005); Smith & Simmons (2012); Silver & Veness (2010); Moss et al. (2023); Muškardin et al. (2024). These strategies address the same broad problem class, but the present work focuses on a different algorithmic question: we isolate the effect of replacing the classical rejection-sampling belief update inside a fixed look-ahead planner by its quantum counterpart. For that reason, the baseline throughout the paper is the classical version of the same algorithm.

Hence, our contribution is complementary to the existing literature on two topics: quantum reinforcement learning, and classical POMDPs. We do not propose a general-purpose quantum solution to all partially observable reinforcement-learning problems. Rather, we identify a specific regime in which quantum rejection sampling can reduce the cost of belief updates, and make explicit both the associated advantages and the limitations through a complexity analysis and numerical experiments.

3. Background

This section assembles the theoretical ingredients that underpin the quantum-enhanced decision-making approach developed in the remainder of the article. We first revisit POMDPs (§3.1), establishing the notation that will carry through the paper and recalling how belief states encode an agent’s uncertainty about latent system dynamics. The discussion then turns to DDNs (§3.2), a Bayesian-network (BN) realisation of POMDPs that exposes the conditional-independence structure required for efficient probabilistic inference. Because our contribution leverages quantum resources, we next summarise the circuit model of quantum computation (§3.4), and the notation used to represent quantum programs. Finally, we outline the *quantum rejection-sampling* algorithm of Low and Chuang, together with the amplitude-amplification techniques that yield a quadratic speed-up for inference in sparse BNs (§3.6). Collectively, these four building blocks provide the conceptual and algorithmic scaffold for the quantum POMDP solver presented in Section 6.

3.1. POMDPS

Partially observable Markov decision processes (POMDPs) are Markov decision processes (MDPs) where the state can’t be directly observed by an agent. The agent, having to select an action $a_t \in \mathcal{A}$, relies instead in an observation $o_t \in \Omega$, obtained from the *sensor model* $P(o_{t+1}|s_{t+1}, a_t)$ of the environment, to describe the

state $s_t \in \mathcal{S}$. Sets, random variables and their values are denoted by calligraphic, uppercase and lowercase letters respectively. The environment evolves over time given its *transition dynamics* $P(s_{t+1}|s_t, a_t)$ and the expected reward is obtained from the *reward function* $\mathbb{E}(r_{t+1}|s_t, a_t)$. These three quantities describe the environment's behaviour and are called the *environment dynamics*.

A prior state distribution $b_0(s) = P(s)$ is used to define the initial belief state. As the agent selects actions and receives observations, it gathers a *history* $h_t = \{a_0, o_1, \dots, a_{t-1}, o_t\}$ of past information it can use to update its uncertainties about the environment. One way of achieving this is by defining a *belief state* $b_t(s) = P(s|h_t, b_0)$, a probability distribution describing the uncertainties of the agent over the environment states, which it can't directly observe. It is also possible to sequentially update this belief state: given an action a_t , an observation o_{t+1} and a previous belief state b_t , the updated belief state b_{t+1} can be obtained using the *belief updating rule*:

$$b_{t+1}(s_{t+1}) = P(s_{t+1}|b_t, a_t, o_{t+1}) \propto P(o_{t+1}|s_{t+1}, a_t) \sum_{s_t \in \mathcal{S}} P(s_{t+1}|s_t, a_t) b_t(s_t) \quad (1)$$

Adopting a more concise notation, denoting by $\mathcal{Z}$ the sensor model (conditional distribution of observations given states and actions), and dropping the subscripts in favor of primes to denote the following state, the belief update rule can be more succinctly expressed as $\tau(b, a, o)(s') \equiv b'(s') \propto \mathcal{Z}_{s'a}^o \sum \mathcal{P}_{ss'}^a b(s)$.

The behaviour of an agent can be characterized by a *policy* $\pi(b, a) = P(a|b)$, a probability distribution over actions given belief states. The behaviour of an agent produces a sequence of rewards, which can be incorporated into a metric, called the return $G_t = \sum_{k=t}^{\infty} \gamma^{k-t} r_{k+1}$, that quantifies the desirability of that sequence. Given the stochastic nature of an environment, the goal of an agent is to maximize the *expected return*, a quantity that is often used in the literature to define the value of both states and actions. The value of a belief state b_t when following a policy π is given by the *state value function* $V^\pi(b)$, while the value of taking action a when in belief state b while following policy π is given by the *action value function* $Q^\pi(b, a)$. They are given by equations equation 2 and 3 respectively.

$$V^\pi(b) = \mathbb{E}_\pi \left(\sum_{k=t}^{\infty} \gamma^{k-t} r_{k+1} \mid b \right) \quad (2) \quad Q^\pi(b, a) = \mathbb{E}_\pi \left(\sum_{k=t}^{\infty} \gamma^{k-t} r_{k+1} \mid b, a \right) \quad (3)$$

A policy π^* is said to be optimal if it yields value functions (also called optimal value functions) no smaller than any other policy's value functions: $V^*(b) = \max_\pi V^\pi(b)$ and $Q^*(b, a) = \max_\pi Q^\pi(b, a)$.

3.2. Dynamic decision networks

A Bayesian network (BN) is a directed, acyclic graph $\mathcal{G} = \{\mathcal{V}, \mathcal{W}\}$ whose vertices $\mathcal{V} = \{X_1, X_2, \dots, X_N\}$ represent random variables (RVs) and the edges $\mathcal{W}$ represent dependencies between them. Each RV contains a conditional probability table (CPT) $P(X_i | \text{par}(X_i))$ describing the probability of occurrence for each value of the RV X_i , given the values of its parent RVs $\text{par}(X_i)$.

A dynamic decision network (DDN) is a BN that is constructed to capture the notion of time dynamics and agency, allowing it to model a POMDP. Its RVs describe the states S_t , actions A_t , observations O_t , and rewards R_t for each time-step t and their dependencies should mimic the environment dynamics of the underlying POMDP (see Figure 1). Their CPTs represent the transition dynamics, the policy, the sensor model, and the reward function, respectively. Moreover, the CPT of the initial state S_0 encodes the prior belief state distribution b_0 .

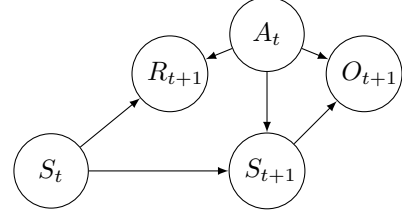


Figure 1: The simplest DDN, with one RV representing each state, action, observation, and reward.

A probability distribution can be extracted from any BN (including DDNs) by using *inference algorithms*, such as *rejection sampling*. Therefore, DDNs can be used to model and extract probabilistic information about the dynamics of an environment.

3.3. Look-ahead algorithm

To approximately solve partially observable RL problems, we consider a lookahead algorithm which extracts a near-optimal action from a finite lookahead tree. The algorithm performs an exhaustive tree-based search of all possible futures of the agent within a given horizon H , enabling the selection of a rational action based on consideration of these future scenarios.

The lookahead tree starts from the current belief state of the agent and expands through the enumeration of actions from the action space $\mathcal{A}$ and observations from the observation space Ω . Each belief state is connected to reward nodes via an edge for each action, and each reward node is connected to a new belief node via an edge for each observation. Following the construction of the tree, the value of each node is iteratively calculated, starting with the leaves and proceeding upwards to the root node. Belief and reward node values are defined using state and action value functions, respectively.

Consider t the current time-step and an horizon H . The value of each leaf reward node is given by the expected reward given their parent belief state b_{t+H} and the action a_{t+H} on the edge to that leaf node:

$$Q^L(b_{t+H}, a_{t+H}) = \mathbb{E}(r_{t+H+1} | b_{t+H}, a_{t+H}) \quad (4)$$

where we annotate the value functions with the superscript L denoting the lookahead, which stops the recursion.

For non-leaf belief nodes, their value is determined by greedily selecting the action that leads to the highest valued reward node, which corresponds to a max operation:

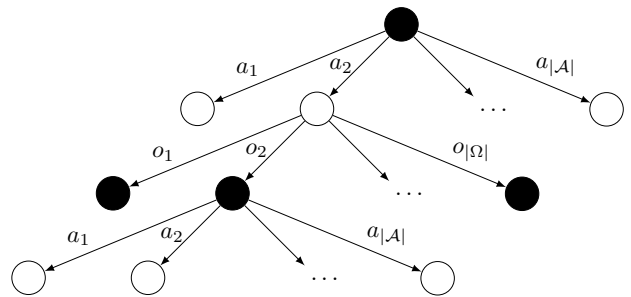


Figure 2: A two-step ($H = 2$) lookahead tree. Belief and reward nodes are represented by black and white circles, respectively.

$$V^L(b_t) = \max_{a_t \in \mathcal{A}} Q^L(b_t, a_t) \quad (5)$$

The value of a non-leaf reward node is calculated using the expected values of its child belief nodes, weighted by the probability of the observation that leads to each child. This accounts for the stochastic nature of observations in the environment, making it imperative to consider all observations in the calculation. Finally, the expected reward for that particular node is also added to the value:

$$Q^L(b_t, a_t) = \mathbb{E}(r_{t+1}|b_t, a_t) + \gamma \sum_{o_{t+1} \in \Omega} P(o_{t+1}|b_t, a_t) V^L(b_{t+1}) \quad (6)$$

where b_{t+1} is obtained via the belief update rule of (1), given the belief state b_t , action a_t and each of the observations o_{t+1} in the summation of (6).

Following the calculation of values in the tree according to the aforementioned expressions, the action that should be taken is the one that maximizes value: $a_t^L = \operatorname{argmax}_{a_t \in \mathcal{A}} Q^L(b_t, a_t)$.

3.4. Quantum computing with quantum circuits

A quantum circuit is a combination of different qubits, quantum gates, and measurements. The qubits encode quantum information that is transformed via the quantum gates to produce some desired result, which is converted to classical information via measurements. In n -qubit quantum systems, a quantum state $|\psi\rangle = (\alpha, \beta)^\top$ exist in a complex *Hilbert space* $\mathcal{H} = (\mathbb{C}^2)^{\otimes n}$ with a unit norm $\langle\psi|\psi\rangle = 1$. It is common practice to use the Dirac notation, with $\langle\psi|$ representing the complex conjugate of $|\psi\rangle$ and $\langle\psi|\phi\rangle$ the inner-product of quantum states.

Quantum gates are represented by unitary operations $U : \mathcal{H} \mapsto \mathcal{H}$ transforming one quantum state into another. Some of the most important single-qubit gates are the *Pauli* X , Y and Z matrices, and the *phase-shift* gate $P(\theta)$, whose matrices are given below.

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix} \quad (7)$$

Single-qubit rotations are generated by the Pauli operators $\mathcal{P} = \{X, Y, Z\}$ through $R_U(\theta) = \exp\left(-\frac{i\theta}{2} U\right)$, $U \in \mathcal{P}$. More generally, any Pauli string $\sigma \in \{I, X, Y, Z\}^{\otimes n}$ defines an n -qubit rotation $R_\sigma(\theta) = \exp\left(-\frac{i\theta}{2} \sigma\right)$, which can entangle the qubits whenever σ acts non-trivially on more than one site. Besides such rotations, the controlled-NOT gate, $\text{CNOT} = \begin{pmatrix} \mathbb{I}_2 & 0 \\ 0 & X_2 \end{pmatrix}$, is the standard primitive for creating entanglement in quantum circuits. Combined with single-qubit rotation gates, it results in a universal gate set for quantum computation.

3.5. Quantum Bayesian networks

A Quantum Bayesian Network (QBN) encodes the joint distribution $P(X_1, \dots, X_N)$ of a classical BN into the amplitudes of an N -qubit state. Let $\mathcal{B}$ denote a state-preparation circuit acting on $|0\rangle^{\otimes N}$ such that

$$|\langle x_1 \cdots x_N | \Psi \rangle|^2 = P(X_1 = x_1, \dots, X_N = x_N), \quad |\Psi\rangle = \mathcal{B} |0\rangle^{\otimes N}. \quad (8)$$

For binary random variables, $\mathcal{B}$ can be constructed with a sequence of uniformly controlled $R_Y(\theta)$ rotations: each node X_i is mapped to a qubit q_i , and for every assignment of the parent variables $\text{parents}(X_i)$ a controlled rotation is applied to q_i . The angle

$$\theta = 2 \arctan \left(\sqrt{\frac{P(X_i = 1 | \text{parents}(X_i) = x_p)}{P(X_i = 0 | \text{parents}(X_i) = x_p)}} \right) \quad (9)$$

implements the corresponding conditional-probability-table (CPT) entry. If $M = \max_i \text{parents}(X_i)$ is the maximum parent count, the computational complexity of the corresponding state preparation circuit scales as $\mathcal{O}(N2^M)$ (Low et al., 2014b). Although this is worse than the classical direct-sampling cost $\mathcal{O}(NM)$, the quantum representation unlocks amplitude-based speed-ups for subsequent inference tasks.

Importantly, the classical overhead associated with parameter calculations does not affect the quantum speed-up. The complexity of computing the angle in equation 9 is $\mathcal{O}(1)$, since the conditional probabilities can be looked up in the CPT of the random variable X_i . Since the number of parent configurations is upper bounded by 2^M , doing this for each of the N qubits requires computing at most $N2^M$ such angles. Thus, these operations can be absorbed into the state preparation cost $\mathcal{O}(N2^M)$ for the complexity considerations.

3.6. Quantum rejection sampling

Given a QBN state (8), one can extract the conditional distribution $P(\mathcal{Q} | \mathcal{E} = e)$ over query variables $\mathcal{Q}$ provided evidence e , by quantum rejection sampling. The joint state is first partitioned as

$$|\Psi\rangle = \sqrt{P(e)} |\mathcal{Q}, e\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \bar{e}\rangle, \quad (10)$$

where $P(e)$ is the evidence probability. An evidence phase-flip operator $\mathcal{S}_e$ inverts the phase of the good subspace $|\mathcal{Q}, e\rangle$, while the Grover diffusion operator $\mathcal{S}_0$ amplifies the states with evidence e . Iterating the amplitude-amplification operator, $G = \mathcal{B} \mathcal{S}_0 \mathcal{B}^\dagger \mathcal{S}_e$, $\mathcal{O}(P(e)^{-1/2})$ times boosts the success amplitude.

For sparse networks, where each node has a small amount of parents M , the exponential factor 2^M in the quantum complexity can be approximated as M , and thus the quadratic quantum speedup is the main distinction between complexities. In these conditions, the total complexity for the quantum algorithm can be written as $\mathcal{O}(N2^M P(e)^{-1/2})$, providing a quadratic improvement in the inference over classical rejection sampling's $\mathcal{O}(NM P(e)^{-1})$ cost Low et al., 2014b. See Appendix A for a complete derivation of the quantum rejection sampling algorithm. The quantum advantage is powered by quantum amplitude amplification, where interference is used to decrease the rejection rate. An amplification operator can be defined using standard quantum operations, and repeatedly applied to boost the acceptance probability. The optimal degree of amplification depends on the probability of the evidence $P(e)$, i.e. the acceptance probability for the sampling. For $P(e) \geq 0.5$, the quantum and classical cases coincide, if using the standard definition of the amplitude amplification operator. However, this operator can be generalized to change this, as described in Brassard et al. (2002). The probability can then be raised to 100% in all cases, provided that $P(e)$ is known.

In practice, this does not hold. In (Low et al., 2014b), an exponential progression proposed in Brassard et al. (2002) is used to remedy this fact. While this method retains the quadratic advantage, the cost offset is not necessarily advantageous in terms of quantum resources. Instead, one may calculate the probability analytically using the Bayesian network (depending on its structure) or employ quantum amplitude estimation or even a rudimentary classical pre-estimation based on a few samples. The estimated value need not be exact; small deviations will manifest as non-optimal, but still high, acceptance rates, without jeopardizing correctness.

Because the same construction applies when the BN is the time-unrolled slice of a dynamic decision network, the QRS procedure serves as the inference engine in the quantum POMDP framework described in Section 4.

4. Quantum-classical POMDP lookahead

In this section, we present quantum Bayesian reinforcement learning (QBRL): a classical model-based RL algorithm that incorporates the quantum rejection sampling algorithm for BNs as a subroutine. We propose quantum circuits for encoding each component of a POMDP, and show they can be used to perform a quantum belief update that is equivalent to the classical one. For proofs, as well as details on the construction of the quantum operators, the reader is referred to Appendix A.

Figure 3 shows the quantum circuit that implements a belief update. Operators $\mathcal{U}(b)$ and $\mathcal{U}(a)$ encode the belief state and action, respectively. The operators $\mathcal{U}_1$, $\mathcal{U}_2$ and $\mathcal{U}_3$ encode the transition dynamics, the sensor model and the reward, respectively. $G^k(o)$ is the k -th power of the amplitude amplification operator for observation o , defined as in subsection 3.6 from an evidence phase flip operator $\mathcal{S}_e$ and the Bayesian network encoding operator $\mathcal{B}$. Each of these operators is defined in Appendix A.

Lemma 4.1 gives the measurement probabilities of this circuit. From this follows theorem 4.2, which determines that the quantum and classical belief updates coincide.

Lemma 4.1. *The probability of measuring $S_{t+1} = s'$ in the quantum circuit of Figure 3 is given by the following expression*

$$\langle s' | \rho(b, a, o) | s' \rangle = \frac{1}{\eta} P(o | s', a) \sum_{s \in \mathcal{S}} P(s' | s, a) b(s) \quad (11)$$

where η is a normalizing constant (defined in appendix A).

Theorem 4.2. *The quantum and classical belief update rules are equivalent for DDNs using rejection sampling.*

Thus, we can replace the classical belief update with a quantum one. Note that this would also apply if we measured the state encoding qubits without applying $G^k(o)$. However, this amplitude amplification step increases the acceptance probability in rejection sampling, yielding a quantum speed up as demonstrated in

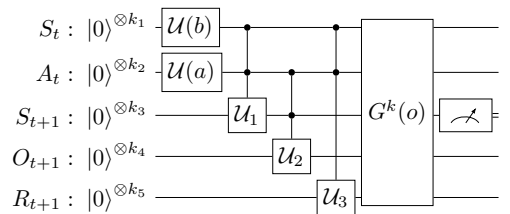


Figure 3: Quantum circuit for a belief update.

Appendix B. The quantum inference subroutine can be used to calculate the expected reward of Equation 5 and Equation 6 and both the observation probabilities and belief updates of Equation 6.

5. Complexity analysis

In the section, we provide our main theoretical results: the computational complexity analysis for both the classical and the quantum-classical POMDP look ahead algorithms. In subsection 5.1 we discuss the possible sources of quantum advantage, after which subsection 5.2 presents our key theorems. We derive a bound on the number of samples for the purely classical case, as well as for our quantum-enhanced version. We additionally derive simplified expressions for the case where the quantum algorithm stands out the most: the belief update dominates the cost of the algorithm.

5.1. MDP disadvantage and POMDP advantage

The quantum rejection sampling inference algorithm can be used to approximate three probability distributions for POMDPs: the reward distribution $P(r_{t+1}|b_t, a_t)$; the observation distribution $P(o_{t+1}|b_t, a_t)$; the belief update $P(s_{t+1}|b_t, a_t, o_{t+1})$. It is assumed that they require a different number of samples n , m and l , respectively, to be approximated.

The evidence of these probability distributions involves the state RV S_t , the action RV A_t , and the observation RV O_{t+1} . Note that from these RVs, only the observation RV is not a root node of the DDN (see Figure 1). As such, the values of the state and action RVs can be directly encoded in their corresponding CPTs before performing inference, ensuring that the evidence of those RVs always matches and never leads to a rejected sample.

For the reward and observation distributions, encoding the evidence implies that a sample can be extracted, without ever being rejected, by simply performing random sampling at each RV. This algorithm is called *direct sampling* and is a simpler and more efficient algorithm when compared to rejection sampling. However, (Low et al., 2014b) shows that the quantum analog of direct sampling is *exponentially slower* than the classical counterpart: the classical and quantum complexities are $\mathcal{O}(NM)$ and $\mathcal{O}(N2^M)$ respectively, where N is the number of RVs in the DDN and M is the maximum number of parents of a RV. This makes the classical algorithm the preferred approach for inferring these probability distributions.

In an MDP, observation RVs do not exist; probability distributions depend only on states and actions. This implies that direct sampling is the best inference algorithm, and thus using its classical version is the most efficient approach. On the other hand, belief updating in POMDPs does need to be performed using rejection sampling, and therefore the quantum rejection sampling algorithm poses an advantage. Likewise, there may be a quantum speed-up for the reward calculation if it requires belief updating.

5.2. Time complexity

We now state the main analytical results of this work, regarding the time complexity of the classical and quantum algorithms. Proofs are provided in Appendix B.

To introduce the notation for the upcoming theorems, there are two important factors that distinguish the complexity for the classical and quantum-classical algorithms. Denoting by Φ the set of all observation probabilities in the lookahead tree, these factors are $c_l = \sum_{p \in \Phi} p^{-1}$ and $q_l = \sum_{p \in \Phi} p^{-\frac{1}{2}}$. Let $\mathcal{S}, \mathcal{A}, \Omega$, and $\mathcal{R}$ denote the sizes of the state, action, observation, and reward spaces, respectively. As previously defined, H denotes the look-ahead size, n and m are the numbers of samples for the reward and observation sampling respectively, N is the number of RVs in the BN, and M is the largest number of parents among all RVs. The complexity of the classical algorithm is then given by Theorem 5.1.

Theorem 5.1. *The computational complexity of the classical lookahead algorithm is given by:*

$$\mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 c_l\right)\right)$$

In contrast, the complexity of the quantum-classical algorithm of section 4 is given by Theorem 5.2.

Theorem 5.2. *If $2^M \approx M$, the computational complexity of the quantum-classical lookahead algorithm is:*

$$\mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 q_l\right)\right)$$

Finally, corollary 5.3 provides simplified expressions under an ideal assumption for the quantum-classical lookahead algorithm: the contribution of the belief updating, the only part with a quantum speedup, is much more computationally expensive than the remaining inferences used in the algorithm.

Corollary 5.3. *If $\frac{1}{q_l} \ll \frac{((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}}$, the expressions of Theorems 5.1 and 5.2 become respectively:*

$$\mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 c_l\right) \quad \mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 q_l\right)$$

In the scenario of Corollary 5.3, the quantum lookahead would witness a speed up of a factor of $\frac{c_l}{q_l} \in [1, q_l]$ (shown in appendix B.2). From this ratio, we can derive upper and lower bounds for the quantum advantage:

$$c_l = q_l^\beta, \quad \beta \in [1, 2] \tag{12}$$

This means that the quantum-classical algorithm has a range of possible speedups: it has the same computational complexity as its classical counterpart in the worst case, and is quadratically faster in the best case. The numerical simulations in section 7 illustrate this variability, demonstrating how this ratio varies depending on the problem and the impact it has on performance.

We stress that the result of equation 12 corresponds to the most advantageous case for the quantum algorithm, which involves two assumptions: the rejection sampling sub-routine dominates the cost, and the maximum in-degree of the Bayesian network is very small. These conditions are formalized as $2^M \approx M$ and $\frac{1}{q_l} \ll \frac{((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}}$, respectively. If they do not hold, the quantum advantage is instead given by:

$$\frac{\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 c_l}{\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 q_l 2^M / M} \tag{13}$$

The second condition is especially restrictive, as the difference between exponential and linear scaling is substantial even for small values of M . However, there is potential for an advantage even if this difference is not negligible, as long it is small. More specifically, if we apply the first condition only, we derive the following condition for quantum advantage:

$$c_l \geq \frac{2^M}{M} q_l \quad (14)$$

In other words, the quantum advantage is penalized exponentially in M , with a break-even point at $c_l = q_l 2^M / M$. As such, there will be a quantum advantage only where the ratio c_l / q_l is large enough to compensate the fast-growing penalty incurred by the quantum algorithm, which is plausible for problems where M is small.

6. Methods

We perform numerical simulations of reinforcement learning problems, with the goal of comparing the performances of our quantum-classical algorithm with the fully classical one. The code is publicly available on GitHub Ramôa (2024). The objective is a practical assessment of what the $\frac{c_l}{q_l}$ speed-up means in practice. This ratio, and thus the magnitude of the quantum advantage in the rejection sampling sub-routine, depends on the problem at hand; thus, so does the effective benefit it entails.

The aim of our empirical study is not exhaustive benchmarking against the full POMDP literature, but a controlled validation of the mechanism predicted by the complexity analysis: when belief updating relies on rejection sampling, quantum rejection sampling can improve the cost-to-quality trade-off. Thus, we evaluate the algorithms under two complementary protocols: fixed query budget and fixed effective-sample budget. We consider two benchmarks, detailed below, which were chosen to expose different operating regimes: the first test case exhibits greater sensitivity to the number of samples, whereas the second has a higher quantum speed-up for the isolated sampling sub-routine. With this, we relate the empirical behavior directly to the key quantity c_l / q_l , and additionally discuss other practical considerations that determine the scenarios where the quantum advantage is most fruitful.

The numerical results we are interested in concern cost-to-benefit ratios. The units of cost are queries, defined as generated (but not accepted) samples¹. The quantum speed-up can be used to improve performance for a given cost; or conversely, to decrease cost (and thus decision making time) for a given performance level. For the purpose of demonstrating these improvements, we consider the time evolution of two quantities. The first one is the cumulative reward, that is, the rewards obtained by the agents, accumulated over time steps. In this case, the number of queries is the same for both algorithms. This experiment illustrates how the quantum algorithm can improve the decision making for a fixed time.

¹In the quantum case, the cost of applying the encoding operator once is equated to the cost of generating one sample, as is standard in the literature Brassard et al. (2002).

The second quantity is the cumulative cost, fixing the average performance requirements (represented by the number of accepted samples). Here, cost is again measured in terms of queries. This experiment illustrates how the quantum algorithm can speed up decision-making for a given performance level, highlighting the resource savings it achieves when replicating its classical counterpart exactly.

For a thorough assessment, we will consider in total 4 types of plots:

1. *Source of quantum advantage.* Cost per sample as a function of the baseline success probability, as in figure 5. This visually demonstrates the quadratic quantum advantage in the rejection sampling sub-routine.
2. *Performance comparison under a fixed query budget.* Cumulative reward difference as a function of the time step, as in figures 6, 7 and 8a. That is, the vertical variable is $\sum_{t=0}^{t_i} r_q - \sum_{t=0}^{t_i} r_c$. This shows the gains of the quantum-classical algorithm as compared to the classical one, for a fixed decision making cost or time. In other words, it isolates the decision quality under equal cost.
3. *Cost comparison under a fixed effective-sample budget.* Cumulative cost difference as a function of the time step, as in figure 8b. That is, the vertical variable is $\sum_{t=0}^{t_i} \text{queries}_q - \sum_{t=0}^{t_i} \text{queries}_c$. This shows the savings of the quantum-classical algorithm as compared to the classical one, for a fixed number of obtained samples and thus quality of the estimates. In other words, it isolates the cost under equal estimator quality.
4. *Cost comparison under a fixed performance level.* Cumulative cost as a function of the cumulative reward, as in figure 9. This is similar to the previous point, with the same vertical variable; but the plot additionally factors in how the difference in the number of samples/quality of the estimates affects the agent’s performance. In other words, it isolates the cost under equal performance.

We run each algorithm 10^2 to 10^3 to obtain averages, and additionally represent the standard deviations among runs as shaded areas where relevant. In the case of the fourth plot category, direct averages cannot be taken due to dispersion in the x-coordinate. To regularize results, we divide the domain into bins, and average the coordinates independently among each bin, to create visually intelligible plots.

Our two test cases are the tiger problem of Kaelbling et al. (1998), and a robot exploration problem, illustrated in figures 4a and 4b, respectively.

The tiger problem is as follows. There are two doors; one leads to a room with a tiger, the other to a room with a treasure. The possible actions are to open one door, the other, or try to listen through the doors. Finding the treasure gives a reward of 5, whereas to find the tiger corresponds to a reward of -10 . Listening has a reward of -1 , and has 15% failure probability. After each action except for listening, the tiger and treasure are randomly reassigned to the doors, so that the decision process can continue indefinitely.

For the robot problem, there are 4 contiguous rooms with a circular layout, one of which contains a treasure. The robot can identify whether it is in the treasure room up to 10% error. At each timestep, the robot can move clockwise or counterclockwise, with a reward of -1 . Alternatively, if it is in the treasure



Figure 4: Illustration of the set ups for the considered test cases.

room, it can pull one of two levers. These levers have different chances of producing the treasure, which brings a reward of 10: 70% and 90%. As a caveat, they otherwise damage it to different degrees, yielding rewards of -5 and -20 for the lever with lower and higher success rates respectively. Pulling a lever in other rooms has no effect. After pulling a lever, the robot is placed back in the initial room.

7. Experimental Results

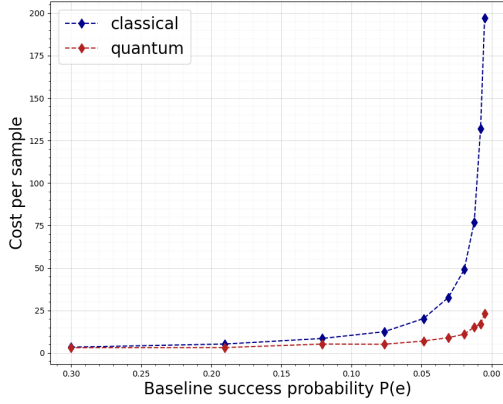
In this section, we numerically test the performance of the QBRL algorithm applied to partially observable reinforcement learning problems, comparing it to the classical algorithm.

The fundamental source of quantum advantage is depicted in figure 5, which shows how the cost increases as the problem gets harder. The cost is higher for lower acceptance probabilities, since obtaining one effective sample requires more samples to be produced and rejected on average.

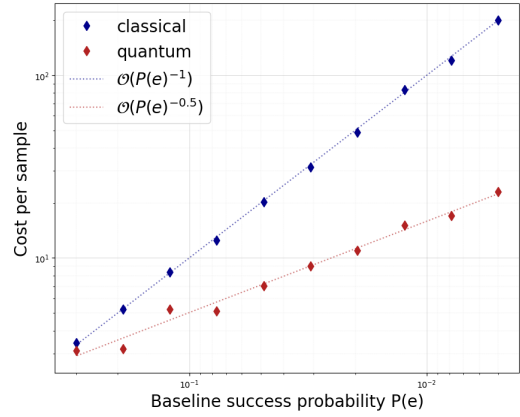
For the classical case, the average cost per sample grows faster as the success probability decreases, as can be seen in figure 5a. Figure 5b clarifies the magnitude of this speed-up: the classical routine has complexity $\mathcal{O}(P(e)^{-1})$, whereas the quantum routine is quadratically faster, with complexity $\mathcal{O}(P(e)^{-0.5})$. This is due to the quantum amplitude amplification process.

In other words, if the acceptance probability becomes 100 times smaller, the classical cost increases by a factor of 100, and the quantum one by a factor of 10. The magnitude of the quantum advantage for a specific instance of rejection sampling depends on the specific values of $P(e)$, which determine the $\frac{c_l}{q_l}$ ratio. The operator generalization mentioned in section 3.4 would smooth out the irregularities in the plot 5a, which are due to the discrete nature of the amplification.

The evolution of cumulative reward difference with the time step for the classical and quantum algorithms under a fixed query cost is presented in figure 6 for various numbers of classical samples ($\{5, 15, 50, 100\}$). The number of effective samples is higher for the quantum algorithm due to the $\frac{c_l}{q_l}$ ratio, which enables a quantum advantage. This advantage depends on the number of classical samples, as well as on the problem. We note the large spread of the values due to the stochastic nature of the problem, which brings large inter-run variability.

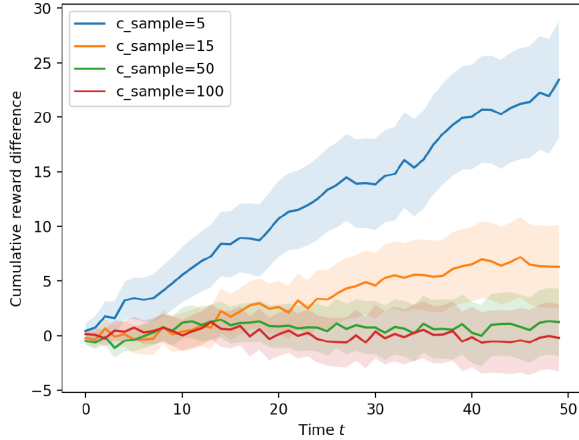


(a) Evolution of the cost per sample with decreasing acceptance probability $P(e)$ for the classical and quantum rejection sampling routines.

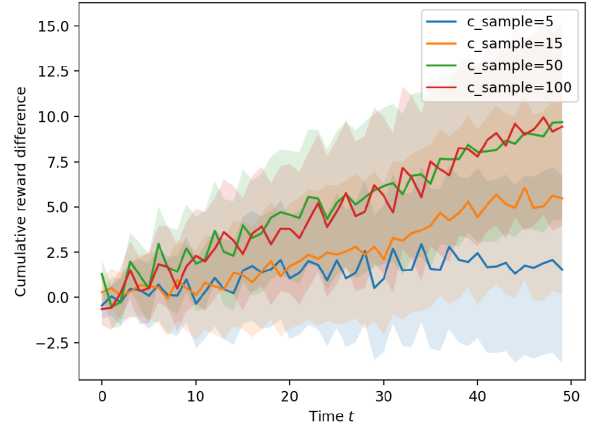


(b) Evolution of the cost per sample with decreasing acceptance probability $P(e)$ for the classical and quantum rejection sampling routines, in a logarithmic scale and juxtaposed with reference lines for the complexity of each.

Figure 5: Quadratic quantum advantage in rejection sampling. The results are averaged over 10^3 runs. The cost per sample corresponds to the average number of queries as defined in the text.



(a) Tiger problem.



(b) Robot problem.

Figure 6: Evolution of the cumulative rewards for the quantum and classical algorithms with time, for the tiger (6a) and robot (figure 6b) problems. The cost is the same for the classical and quantum algorithms, but the latter may obtain higher rewards, depending on the number of samples.

In the tiger problem, the quantum reward advantage peaks for fewer classical samples: the decision-making is especially affected by low sample numbers for this problem, making increases in effective sample numbers particularly critical. This highlights the potential impact of the quantum speed-up in cases where minimal resource usage is pivotal, such as when decisions must be fast or lightweight. For the robot problem, the advantage also stalls after a certain number of samples, as the benefit of extra samples is saturated; however, this number is higher.

We now focus on the sample counts that maximize the reward difference (5 and 50 for the tiger and robot

problems respectively), to highlight the most promising scenarios for quantum advantage. The evolution of the classical and quantum cumulative rewards is shown independently in figure 7, and quantitative results and specifications for each test case are given in table 1. The quantum algorithm achieved a 94% and 8% improvement in the cumulative reward after 50% time-steps for the tiger and robot problems respectively. The smaller gains for the latter problem are related to the previous observation that the benefit of extra samples hits a plateau.

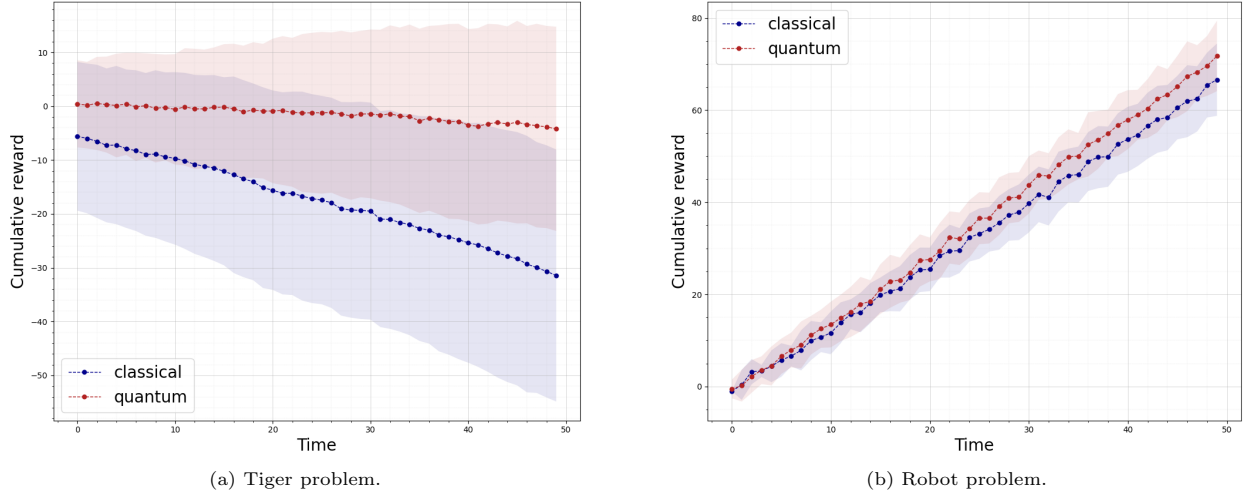
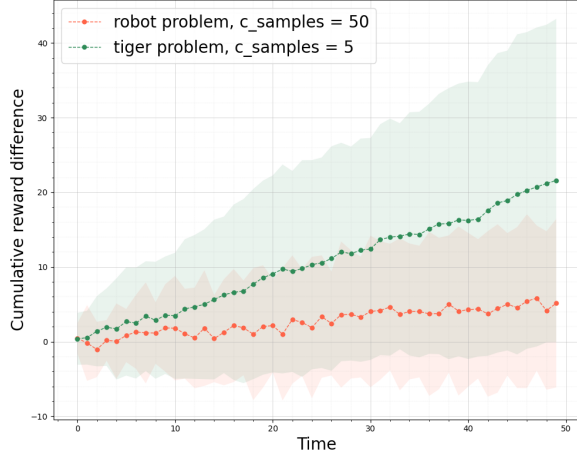


Figure 7: Evolution of the cumulative rewards for the quantum and classical algorithms with time, for the tiger (7a) and robot (figure 7b) problems. The cost is the same for the classical and quantum algorithms, but the quantum ones obtain higher rewards.

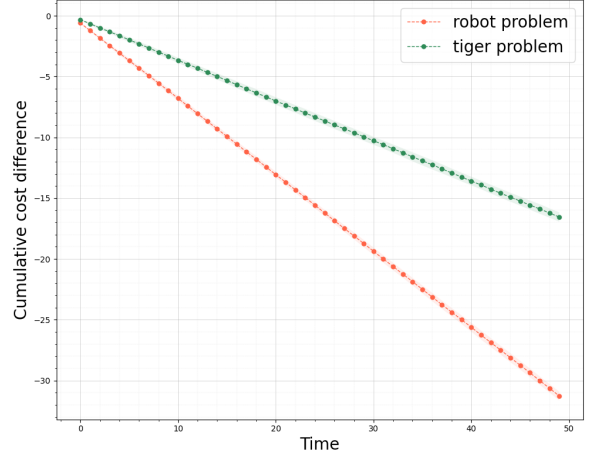
Case	$ \mathcal{S} $	$ \mathcal{A} $	$ \Omega $	$ \mathcal{R} $	H	Samples	c_l/q_l	Final reward	
								Classical	Quantum
Tiger	2	3	2	3	2	5	1.5	-31	-4
Robot	4	4	2	4	2	50	2.7	66	69

Table 1: Specifications and final results for each of the test cases. The final reward was calculated after 50 time-steps. The quantum algorithm outperforms the classical one in both cases, with a greater margin in the tiger problem.

To better interpret the results, further numerical results for these two problems are presented in figure 8. These plots evidence the differences between the performances of the quantum and classical agents for each of the problems. Figure 8a shows how the cumulative reward difference between quantum and classical algorithms evolves over time, illustrating how the quantum-enhanced algorithm improves performance without increasing cost - which is again fixed. It is clear that the magnitude of this improvement is problem-dependent. While the quantum case presents a faster-than-classical increase in the rewards for both problems, the difference is especially marked for the tiger problem, in agreement with figure 7.



(a) Evolution of the cumulative reward difference between the quantum and classical cases with time. The cost is the same for the classical and quantum algorithms. The quantum approaches become increasingly advantageous with the time.



(b) Evolution of the cumulative difference in cost between the quantum and classical cases with time. The expected performance is the same for the classical and quantum algorithms, but the cost is lower for the latter, and the difference increases with the time.

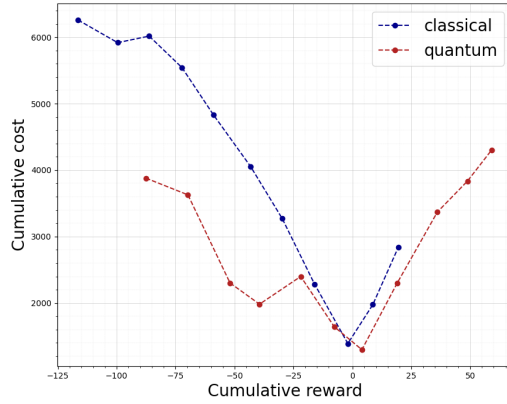
Figure 8: Numerical assessment of the gains obtained by the quantum algorithm. It improves decision-making (figure 8a, or equivalently, reduces the computational cost (figure 8b). Mean results over 100 runs are presented, with shaded areas marking one standard deviation. The look-ahead horizon is $H = 2$ in both cases, and 250 samples are used to approximate the reward distribution.

Figure 8b shows the cost difference over time. In this case, we disregard the number of samples, as it is a multiplicative factor; different values would hinder the visual analysis. This plot showcases how the quantum algorithm spends less resources for the same returns - the number of effective samples, and thus the quality of the decision making, being held constant. This quantity has a smoother evolution, owing to the fact that we are considering expected sample counts rather than effective outcomes. While the latter is related to the former, it is subject to noise, which is injected by the sampling process. In this case, neither of the axes is directly related to performance; as per figure 7a, an increase in time and cost may mean a decrease in rewards. Here, we are merely considering how the cost varies with the number of algorithm steps.

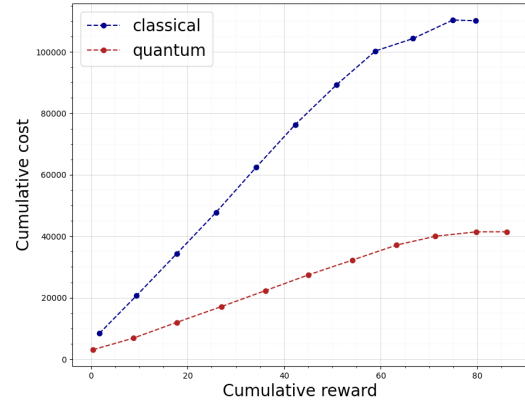
For this metric, the robot problem shows a greater impact of the quantum routine, unlike before. This means that the $\frac{c_l}{q_l}$ ratio is larger. Clearly, this does not directly manifest as improved returns in practice. The reason is that this ratio quantifies the improvement in sampling efficiency, but not how this sampling affects performance. Some problems are more sensitive to sampling variations, similarly to how some numbers of samples show greater quantum advantage. In some cases, a small increase in the number of samples can bring significant returns, while others may require a larger difference to show a comparable effect.

Figure 9 shows how the average cost varies with the achieved rewards. This processing effectively injects noise back into figure 8b, by considering the effective outcome rather than the expected performance.

In this case, taking averages is not straightforward, as the rewards take a very large range of possible values, each of which does not necessarily have a substantial number of data points. As such, we instead bin



(a) Tiger problem.



(b) Robot problem.

Figure 9: Relationship between the achieved reward and the cost (number of queries) for the classical and quantum algorithms, for the tiger (figure 9a) and robot (9b) problems. Results were processed by binning and averaging over 100 runs..

the data, and average each coordinate independently within bins. This is why the ranges of x-coordinates are different for each algorithm. In the limit of infinitesimal bins, this reduces to taking averages. We empirically found that bins of non-null width still produce sensible results for the cases tested herein.

Figures 9a and 9b further elucidate the performance differences between the tiger and robot examples. In the tiger problem, there is a larger spread of rewards, namely over negative values. The classical algorithm has runs with more negative cumulative rewards, which are associated with large costs. This is not unexpected, as large absolute values of reward are associated with larger costs regardless of sign: accumulating them through time steps requires more updates.

Furthermore, the classical algorithm never achieves rewards as good as the best performing quantum runs, even when expending more resources. This is because the agent’s poor performance tends to contribute negatively towards the reward as the process advances; positive cumulative rewards are statistical deviations from the expected behavior, and have low costs because those deviations are more likely to occur early.

In the robot problem, figure 9b, both quantum and classical algorithms display more regular behavior. Higher costs are associated with higher cumulative rewards within each strategy, but the quantum algorithm spends less resources even while performing better. These results, along with those of figure 8a, suggest that the greater advantage in the tiger problem is due to especially poor performance of the classical algorithm. The classical results are shifted towards more negative rewards, due to the small number of samples. The robot problem shows a smaller discrepancy in final rewards, but a larger discrepancy in cost. This occurs because the number of samples, which is larger for this case, is a multiplicative factor in the cost difference.

Another salient point for this discussion are cost considerations beyond the concept of a query. In section 5, we pointed out that the quantum algorithm is exponentially slower in the maximum number of parents of any random variable M . This cost is absorbed into each query, since the operator encoding

the Bayesian network, which has an exponential dependence on M , is necessary to produce the quantum samples. Thus, when zooming in further than queries and looking at elementary operations, this cost may cancel out the quantum advantage for large enough M . This consideration harmonizes the analytical and numerical discussions. We note that the complexity analysis considers the worst case scenario; in practice, the full structure of the tree, and not just the largest among all parent counts, will determine this cost.

8. Conclusion

Reinforcement learning underpins numerous achievements in AI, but its application to partially observable environments remains a major challenge. Due to the added complexity brought by the extra probabilistic layer, such applications often struggle with computational bottlenecks. Our work demonstrates how this problem can benefit from quantum resources, showcasing the sources and limitations of the quantum speedup. Specifically, we introduce a hybrid quantum-classical reinforcement learning algorithm that leverages quantum rejection sampling for belief updates within sparse Bayesian networks, obtaining a quantum speed-up for this key sub-routine.

This speed-up can range between null and quadratic, depending on the problem - more specifically, on the structure of the Bayesian network used to model the Markov decision process (largest in-degree among all nodes). The quantum rejection sampling sub-routine can produce samples at lower cost than its classical counterpart, with a complexity reduction that depends on the problem setting. We demonstrate this through a rigorous, oracle-free computational complexity analysis, producing bounds on the classical and hybrid algorithms. We stress that our algorithm is advantageous only when the decision making occurs in a partially observable environment which can be modeled as a sparse Bayesian network.

To facilitate adoption, we provide all code necessary to reproduce and extend our results, as well as tutorials. In practice, the gains of the quantum approach also depend on the impact of extra samples on performance. Our algorithm is likely to stand out in cases where decisions must be made with limited resources, the performance is most sensitive to variations in the number of samples used for the belief update, and/or the magnitude of the quantum speed-up is highest. We demonstrate this via numerical simulations for different problems, where the quantum resources improved performance to varying degrees. Our simulations illustrate the practical benefits of the quantum algorithm, which stand out in sample-constrained regimes and for environments where belief accuracy significantly affects the quality of the decisions.

An interesting direction for future work is the analysis of model-free approaches to reinforcement learning, where the environment description must itself be learned by observation. Such an approach could use learning algorithms to construct a dynamic decision network based on data, in which case the algorithm we presented could be utilized.

Another possible line of research is the development of a quantum algorithm for choosing an action based on the look ahead tree. This could be done by encoding paths as quantum states, and expected rewards as their amplitudes. This idea is similar to de Oliveira & Barbosa (2021), where a quantum unitary operator

that weights probability amplitudes according to their utility is developed. However, this operator assumes a static utility, and would here need to be generalized for the case of sequential decision making where the utility is a sequence of rewards.

Finally, a natural next step is testing our algorithm in real world scenarios, rather than the toy problems we studied. While these serve as illustrative test cases and a starting point for discussion, it would be interesting to assess the practical benefits for realistic applications. Such demonstrations will become both more relevant and more feasible as quantum devices are scaled up.

References

- Bai, Y., Gao, Y., Wan, R., Zhang, S., & Song, R. (2024). A review of reinforcement learning in financial applications. *Annual Review of Statistics and Its Application*, . doi:10.1146/annurev-statistics-112723-034423.
- Bergholm, V., Vartiainen, J. J., Möttönen, M., & Salomaa, M. M. (2005). Quantum circuits with uniformly controlled one-qubit gates. *Physical Review A*, 71. doi:10.1103/physreva.71.052330.
- Borujeni, S. E., & Nannapaneni, S. (2021). Modeling time-dependent systems using dynamic quantum Bayesian networks. *arXiv preprint arXiv:2107.00713*, . arXiv:2107.00713.
- Borujeni, S. E., Nannapaneni, S., Nguyen, N. H., Behrman, E. C., & Steck, J. E. (2021). Quantum circuit representation of Bayesian networks. *Expert Systems with Applications*, 176, 114768.
- Brassard, G., Hoyer, P., Mosca, M., & Tapp, A. (2002). Quantum amplitude amplification and estimation. *Contemporary Mathematics*, 305, 53–74.
- Bukov, M., Day, A. G., Sels, D., Weinberg, P., Polkovnikov, A., & Mehta, P. (2018). Reinforcement learning in different phases of quantum control. *Physical Review X*, 8. URL: <http://dx.doi.org/10.1103/physrevx.8.031086>. doi:10.1103/physrevx.8.031086.
- Chen, C.-L., & Dong, D.-Y. (2008). Superposition-inspired reinforcement learning and quantum reinforcement learning. In *Reinforcement Learning*. I-Tech Education and Publishing. URL: <http://dx.doi.org/10.5772/5275>. doi:10.5772/5275.
- Cherrat, E. A., Kerenidis, I., & Prakash, A. (2023a). Quantum reinforcement learning via policy iteration. *Quantum Machine Intelligence*, 5, 30. doi:10.1007/s42484-023-00116-1.
- Cherrat, E. A., Raj, S., Kerenidis, I., Shekhar, A., Wood, B., Dee, J., Chakrabarti, S., Chen, R., Herman, D., Hu, S., Minssen, P., Shaydulin, R., Sun, Y., Yalovetzky, R., & Pistoia, M. (2023b). Quantum Deep Hedging. doi:10.48550/arXiv.2303.16585. arXiv:2303.16585.

- de Oliveira, M., & Barbosa, L. S. (2021). Quantum Bayesian decision-making. *Foundations of Science*, (pp. 1–21).
- Dunjko, V., Liu, Y.-K., Wu, X., & Taylor, J. M. (2018). Exponential improvements for quantum-accessible reinforcement learning. doi:10.48550/arXiv.1710.11160. arXiv:1710.11160.
- Ernst, J. O., Chatterjee, A., Franzmeyer, T., & Kuhn, A. (2025). Reinforcement learning for quantum control under physical constraints. URL: <https://arxiv.org/abs/2501.14372>. arXiv:2501.14372.
- Fiderer, L. J., Schuff, J., & Braun, D. (2021). Neural-network heuristics for adaptive bayesian quantum estimation. *PRX Quantum*, 2, 020303. doi:10.1103/PRXQuantum.2.020303.
- Ganguly, B., Xu, Y., & Aggarwal, V. (2024). Quantum Speedups in Regret Analysis of Infinite Horizon Average-Reward Markov Decision Processes. doi:10.48550/arXiv.2310.11684. arXiv:2310.11684.
- Gao, X., Anschuetz, E. R., Wang, S.-T., Cirac, J. I., & Lukin, M. D. (2022). Enhancing generative models via quantum correlations. *Physical Review X*, 12, 021037.
- Ghavamzadeh, M., Mannor, S., Pineau, J., & Tamar, A. (2015). Bayesian reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 8, 359–483. URL: <http://dx.doi.org/10.1561/22000000049>. doi:10.1561/22000000049.
- Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing Stoc '96* (pp. 212–219). New York, NY, USA: Association for Computing Machinery. doi:10.1145/237814.237866.
- Herman, E., & Strang, G. (2016). *Calculus Volume 2*. OpenStax.
- Jerbi, S., Cornelissen, A., Ozols, M., & Dunjko, V. (2022). Quantum policy gradient algorithms. doi:10.4230/LIPIcs.TQC.2023.13. arXiv:2212.09328.
- Jerbi, S., Gyurik, C., Marshall, S., Briegel, H., & Dunjko, V. (2021). Parametrized Quantum Policies for Reinforcement Learning. In *Advances in Neural Information Processing Systems* (pp. 28362–28375). Curran Associates, Inc. volume 34.
- Kaelbling, L. P., Littman, M. L., & Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101, 99–134.
- Kaldari, J., Tariq, S., Al-Kuwari, S., Chen, S. Y.-C., Chatzinotas, S., & Shin, H. (2025). Quantum reinforcement learning: Recent advances and future directions. URL: <https://arxiv.org/abs/2510.14595>. arXiv:2510.14595.
- Liu, X.-H., Xue, Z., Pang, J.-C., Jiang, S., Xu, F., & Yu, Y. (2021). Regret Minimization Experience Replay in Off-Policy Reinforcement Learning. *Advances in Neural Information Processing Systems*, . doi:10.48550/ARXIV.2105.07253.

- Low, G. H., Yoder, T. J., & Chuang, I. L. (2014a). Quantum inference on Bayesian networks. *Physical Review A*, 89, 062315.
- Low, G. H., Yoder, T. J., & Chuang, I. L. (2014b). Quantum Inference on Bayesian Networks. *Physical Review A*, 89, 062315. doi:10.1103/PhysRevA.89.062315. arXiv:1402.7359.
- Meyer, N., Ufrecht, C., Periyasamy, M., Scherer, D. D., Plinge, A., & Mutschler, C. (2024). A survey on quantum reinforcement learning. arXiv:2211.03464.
- Moss, R. J., Corso, A., Caers, J., & Kochenderfer, M. J. (2023). Betazero: Belief-state planning for long-horizon pomdps using learned approximations. *arXiv preprint arXiv:2306.00249*, .
- Muškardin, E., Tappler, M., Aichernig, B. K., & Pill, I. (2024). Reinforcement Learning Under Partial Observability Guided by Learned Environment Models. In P. Herber, & A. Wijs (Eds.), *Integrated Formal Methods* (pp. 257–276). Cham: Springer Nature Switzerland. doi:10.1007/978-3-031-47705-8_14.
- Nian, R., Liu, J., & Huang, B. (2020). A review On reinforcement learning: Introduction and applications in industrial process control. *Computers and Chemical Engineering*, 139, 106886. doi:10.1016/j.compchemeng.2020.106886.
- Niu, M. Y., Boixo, S., Smelyanskiy, V. N., & Neven, H. (2019). Universal quantum control through deep reinforcement learning. In *AIAA Scitech 2019 Forum*. American Institute of Aeronautics and Astronautics. URL: <http://dx.doi.org/10.2514/6.2019-0954>. doi:10.2514/6.2019-0954.
- Papadimitriou, C. H., & Tsitsiklis, J. N. (1987). The Complexity of Markov Decision Processes. *Mathematics of Operations Research*, 12, 441–450. arXiv:3689975.
- Pineau, J., Gordon, G., & Thrun, S. (2003). Point-based value iteration: An anytime algorithm for pomdps. In *IJCAI*.
- Ramôa, A. (2024). Quantum bayesian reinforcement learning. URL: <https://github.com/alexandra-frca/QBRL>.
- Sannia, A., Giordano, A., Gullo, N. L., Mastroianni, C., & Plastina, F. (2023). A hybrid classical-quantum approach to speed-up Q-learning. *Scientific Reports*, 13. doi:10.1038/s41598-023-30990-5.
- Seneta, E. (1992). On the history of the Strong Law of Large Numbers and Boole’s inequality. *Historia Mathematica*, 19, 24–39. doi:10.1016/0315-0860(92)90053-E.
- Sidford, A., Wang, M., Wu, X., Yang, L. F., & Ye, Y. (2018). Near-optimal time and sample complexities for solving discounted Markov decision process with a generative model. *arXiv preprint arXiv:1806.01492*, . arXiv:1806.01492.

- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M. et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529, 484–489.
- Silver, D., & Veness, J. (2010). Partially observable monte-carlo planning (pomcp). In *UAI*.
- Skolik, A., Jerbi, S., & Dunjko, V. (2022). Quantum agents in the Gym: A variational quantum algorithm for deep Q-learning. *Quantum*, 6, 720. doi:10.22331/q-2022-05-24-720. arXiv:2103.15084.
- Smith, T., & Simmons, R. (2012). Heuristic search value iteration for pomdps (hsvi). In *UAI*.
- Spaan, M. T., & Vlassis, N. (2005). Perseus: Randomized point-based value iteration for pomdps. *Journal of Artificial Intelligence Research*, 24, 195–220.
- Wang, D., Sundaram, A., Kothari, R., Kapoor, A., & Roetteler, M. (2021). Quantum algorithms for reinforcement learning with a generative model. In *International Conference on Machine Learning* (pp. 10916–10926). PMLR.
- Wang, S., Zhang, S., Zhang, J., Hu, R., Li, X., Zhang, T., Li, J., Wu, F., Wang, G., & Hovy, E. (2025). Reinforcement Learning Enhanced LLMs: A Survey. doi:10.48550/arXiv.2412.10400. arXiv:2412.10400.
- Xu, Y., & Aggarwal, V. (2025). Accelerating Quantum Reinforcement Learning with a Quantum Natural Policy Gradient Based Approach. doi:10.48550/arXiv.2501.16243. arXiv:2501.16243.
- Yu, H., Zhao, X., & Chen, C. (2025). Quantum-inspired reinforcement learning for quantum control. *IEEE Transactions on Control Systems Technology*, 33, 61–76. doi:10.1109/TCST.2024.3437142.

A. Quantum belief update

Here we detail how to use quantum resources for the quantum belief update, getting the quadratic quantum speed-up for rejection sampling.

A.1. Quantum Bayesian networks

Encoding a BN representing a probability distribution $P(X_1, X_2, \dots, X_N)$ in a quantum circuit means being able to construct a quantum state $|\Psi\rangle = \mathcal{B}|0\rangle^{\otimes N}$ whose qubits represent the RVs.

$$|\langle x_1 x_2 \dots x_N | \Psi \rangle|^2 = P(X_1 = x_1, X_2 = x_2, \dots, X_N = x_N) \quad (\text{A.1})$$

Once the circuit is properly encoded, sampling from the BN joint distribution in the quantum setting is equivalent to performing measurements in the qubits of the circuit. For simplicity, we assume binary RVs. For a description of this process for arbitrary discrete RVs, refer to Borujeni et al. (2021).

Consider a quantum circuit with N qubits q_i , each representing a binary RV X_i . Each entry of the CPT encodes a probability $P(X_i = x_i | \text{parents}(X_i) = x_p)$ of the occurrence of a value x_i in a RV X_i , given a value

x_p for the RV's parents, $\text{parents}(X_i)$. In the quantum setting, for each RV X_i , a controlled $R_Y(\theta)$ rotation gate is applied for each possible value x_p its parent RVs can take, where the control qubits of this gate are the qubits q_i representing RVs parents (X_i). This ensures that any RVs that share an edge in the BN are entangled in the quantum circuit, expressing the information they share.

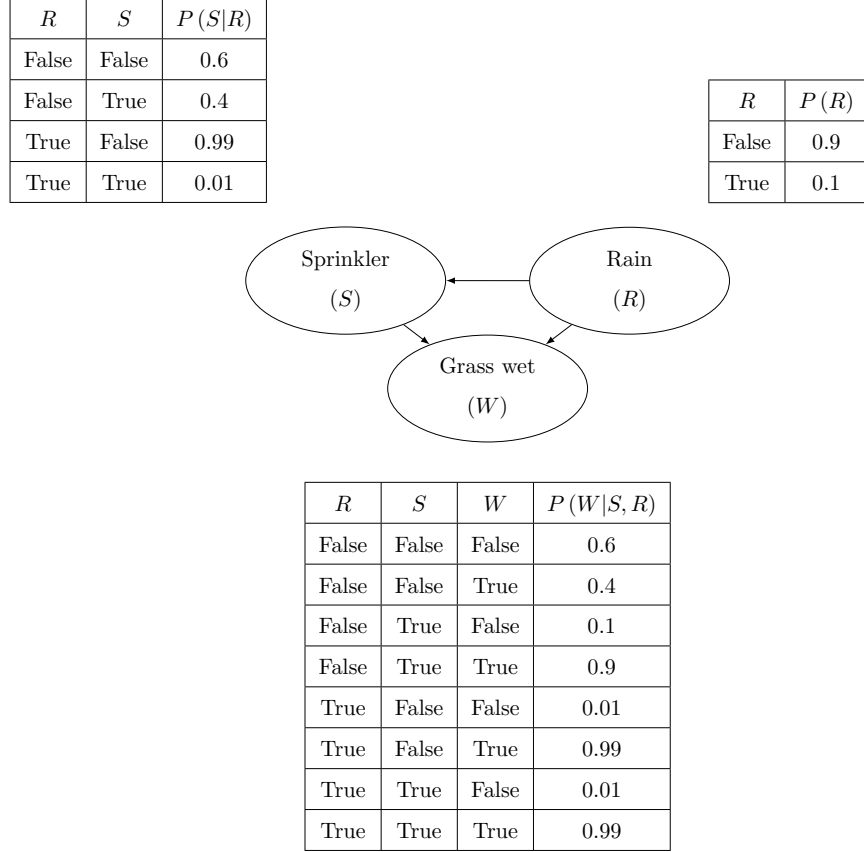


Figure A.10: A BN over three binary random variables

Consider the BN from Figure A.10 as an example. To encode it in a quantum circuit, the following gates have to be applied (see Figure A.11):

- For the variable Rain R , with no parents, an uncontrolled rotation gate is applied.
- For Sprinkler S , with R as a parent, two rotation gates are applied, controlled by the qubit representing R . The first one represents the case when $|R\rangle = |0\rangle$ ², and the second one when $|R\rangle = |1\rangle$.
- For WetGrass W , which has two parents, four rotation gates, controlled by both $|R\rangle$ and $|S\rangle$, are

²For $|R\rangle = |0\rangle$, an X gate must be applied *before* and *after* the control operation on qubit $|R\rangle$. This is because a controlled gate, by definition, only changes the target state when its control qubit is in state $|1\rangle$, but in this case, the opposite effect is wanted. Therefore, the first X gate flips state $|R\rangle$ before the control operation, and the second X gate returns the $|R\rangle$ state to the way it was before the first X gate was applied, leaving it unchanged.

needed. The first one for when $|RS\rangle = |00\rangle$, the second for when $|RS\rangle = |01\rangle$, the third for $|RS\rangle = |10\rangle$ and a final one for $|RS\rangle = |11\rangle$.

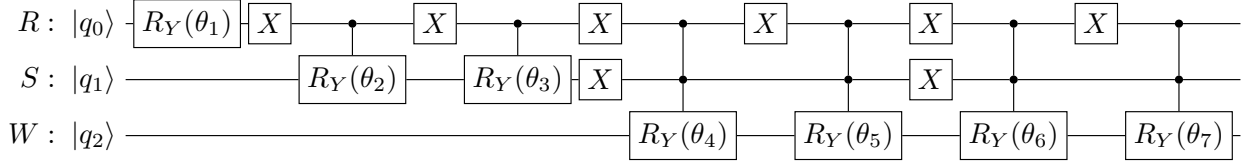


Figure A.11: Quantum circuit encoding the BN of Figure A.10.

Finally, the only missing step for the encoding is knowing the angle for each rotation gate. Consider the controlled rotation gate for a binary RV X_i with value x_p for its parents. The angle of rotation θ is given by:

$$\theta = 2 \arctan \left(\sqrt{\frac{P(X_i = 1 | \text{parents}(X_i) = x_p)}{P(X_i = 0 | \text{parents}(X_i) = x_p)}} \right) \quad (9)$$

As Low et al. (2014b) details, this encoding can be done with a complexity $\mathcal{O}(N2^M)$, where N represents the number of RVs in the BN and M represents the greatest number of parents of any RV. Given that this encoding allows the sampling from the joint probability distribution, it can be directly compared to direct sampling, which has a complexity of $\mathcal{O}(NM)$. The quantum encoding is therefore generally slower than direct sampling, due to the need to include at most M qubit uniformly controlled rotations, each resulting in a decomposition of $\mathcal{O}(2^M)$ elementary gates Bergholm et al. (2005).

A.2. The evidence phase flip operator

Having explained the state preparation circuit $\mathcal{B}$, the missing part of the algorithm is the oracle operator, which, in this particular example, is the evidence phase flip operator $\mathcal{S}_e$. In quantum Bayesian inference, the good states are states where the evidence qubits $\mathcal{E}$ match the evidence value e of the conditional distribution $P(\mathcal{Q} | \mathcal{E} = e)$ that is to be inferred.

In this sense, the amplitude amplification technique turns the joint probability distribution of the BN into the aforementioned conditional distribution by decreasing the amplitudes of states with incorrect evidence $\mathcal{E} \neq e$ and amplifying the amplitudes of states with the right evidence $\mathcal{E} = e$. Formally, the resulting state $|\Psi\rangle = \mathcal{B}|0\rangle^{\otimes N}$ of the BN quantum encoding can be expressed as:

$$|\Psi\rangle = \sqrt{P(e)} |\mathcal{Q}, e\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \bar{e}\rangle \quad (\text{A.2})$$

where $|\mathcal{Q}, e\rangle$ represents the quantum states with the correct evidence e , $|\mathcal{Q}, \bar{e}\rangle$ the quantum states with the wrong evidence and $P(e)$ the probability of occurrence of evidence e . Therefore, the effect of $\mathcal{S}_e$ on this quantum state is given by:

$$\mathcal{S}_e |\Psi\rangle = -\sqrt{P(e)} |\mathcal{Q}, e\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \bar{e}\rangle \quad (\text{A.3})$$

Let $e = e_1 e_2 \dots e_k$ represent the evidence bit string, with $e_i \in \mathbb{B}$. Constructing this operator requires the help of two other operators:

- The operator $\mathcal{X}_i = \mathbb{I} \otimes \dots \otimes X \otimes \dots \otimes \mathbb{I}$ (where $\mathbb{I}$ is the identity), which flips the i -th evidence qubit by applying an X gate to it and leaving every other qubit unchanged,
- The controlled phase operator $P_{1\dots k}$, which flips the phase of a quantum state if all evidence qubits are in state $|1\rangle$.

With the above two operators defined, the evidence phase flip operator $\mathcal{S}_e$ can be defined as follows:

$$\mathcal{S}_e = \mathcal{F} P_{1\dots k} \mathcal{F}, \quad \mathcal{F} = \prod_{i=1}^k \mathcal{X}_i^{1-e_i} \quad (\text{A.4})$$

Consider once more the example of Figure A.10 with evidence qubits R and W and values 1 and 0, respectively, to infer the probability distribution $P(S|R=1, W=0)$. The evidence phase flip operator for this specific case is represented in Figure A.12

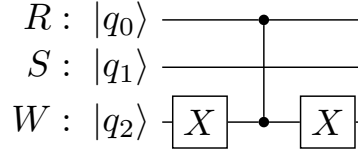


Figure A.12: Quantum circuit for the evidence phase flip operator $\mathcal{S}_e$ for the BN of Figure A.10 with evidences $R = 1$ and $W = 0$.

Notice from Equation (A.4) that the quantum operator $\mathcal{F}$ only flips the evidence qubits with evidence $e_i = 0$, otherwise it leaves them unchanged. This allows for the operator $\mathcal{S}_e$ to behave as expressed by Equation (A.3). To show this, let $|\mathcal{Q}, \tilde{e}\rangle = \mathcal{F} |\mathcal{Q}, \bar{e}\rangle$, where $|\tilde{e}\rangle \neq |1\rangle^{\otimes k}$, represent the application of the quantum operator $\mathcal{F}$ to a superposition of quantum states whose evidence does not match e . Then:

$$\begin{aligned} \mathcal{S}_e |\Psi\rangle &= \mathcal{F} P_{1\dots k} \mathcal{F} \left(\sqrt{P(e)} |\mathcal{Q}, e\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \bar{e}\rangle \right) \\ &= \mathcal{F} P_{1\dots k} \left(\sqrt{P(e)} |\mathcal{Q}, 1\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \tilde{e}\rangle \right) \\ &= \mathcal{F} \left(-\sqrt{P(e)} |\mathcal{Q}, 1\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \tilde{e}\rangle \right) \\ &= -\sqrt{P(e)} |\mathcal{Q}, e\rangle + \sqrt{1 - P(e)} |\mathcal{Q}, \bar{e}\rangle \end{aligned} \quad (\text{A.5})$$

This operator, together with the BN encoding, leads to an amplitude amplification operator

$$G = \mathcal{B} \mathcal{S}_0 \mathcal{B}^\dagger \mathcal{S}_e \quad (\text{A.6})$$

making the quantum inference in BNs quadratically faster than in the classical case, provided that the BN has a small maximum number of parents M :

Contrary to many other speedups in quantum algorithms, this particular one does not require any oracle queries. As the authors state, the complexity of this algorithm is completely determined by counting primitive quantum operations. Another important mention is that this algorithm can also be applied to DBNs, as Borujeni & Nannapaneni (2021) shows, by also encoding observation nodes in the quantum circuit and

	Classical	Quantum
Complexity	$\mathcal{O}(NMP(e)^{-1})$	$\mathcal{O}(N2^M P(e)^{-\frac{1}{2}})$

Table A.2: Comparison between classical and quantum complexity for rejection sampling inference in BNs, taken from Low et al. (2014b).

considering a finite number of time-slices of the network at a time, performing inference over those finite slices. The only difference concerning performing inference in a regular BN is their relative size. Moreover, the same concept can be applied to DDNs by also encoding action and reward nodes into qubits, allowing this inference to be performed in the context of solving RL problems.

A.3. Quantum belief update

In this subsection, we break down the Bayesian network encoding operator $\mathcal{B}$ introduced in section 3.5 for the particular case of a dynamic decision network encoding a POMDP.

Classically, a belief update is performed according to:

$$\tau_c(b, a, o)(s') \equiv P(s'|o, a, b) \propto P(o|s, a) \sum_{s \in \mathcal{S}} P(s'|a, s) b(s) \quad (\text{A.7})$$

A belief update can be performed in a quantum setting by encoding the DDN in Figure 3 with the right action $A_t = a$, a superposition on RV S_t according to its belief-state b and performing amplitude amplification on observation $O_{t+1} = o$. If $\tau_q(b, a, o)$ represents the quantum belief update, then the following reasoning seeks to show that the following equation holds:

$$\tau_q(b, a, o)(s') = \tau_c(b, a, o)(s'), \quad \forall s' \in \mathcal{S} \quad (\text{A.8})$$

To prove this claim, first consider the quantum circuit corresponding to a belief update of Figure 3. The various $\mathcal{U}$ gates perform the encoding of the CPTs of the DDN, while the operator $G^k(o)$ performs the amplitude amplification of the quantum states with the observation o . To formally define these operators, a number of definitions are required.

To encode the belief state of the DDN, the operator $\mathcal{U}(b)$ is applied as in Definition A.1:

Definition A.1. *The quantum operator $\mathcal{U}(b)$ encodes belief state b into the qubits of S_t :*

$$\mathcal{U}(b) |0\rangle^{\otimes k_1} = \sum_{s \in \mathcal{S}} \sqrt{b(s)} |s\rangle$$

The quantum operator $\mathcal{U}(a)$ is used to encode action a into the quantum circuit of the DDN, as in Definition A.2:

Definition A.2. *The quantum operator $\mathcal{U}(a)$ encodes action a into qubits representing A_t , such that:*

$$\mathcal{U}(a) |0\rangle^{\otimes k_2} = |a\rangle$$

To encode the transition dynamics of the DDN, a controlled operator, $\mathcal{U}_1$ is applied to the quantum circuit, as in Definition A.3:

Definition A.3. *The operator $\mathcal{U}_1$ is a controlled quantum operator that encodes the transition dynamics $P(S_{t+1}|s_t, a_t)$ on the qubits of RV S_{t+1} , based on the values s_t and a_t of RVs S_t and A_t , respectively:*

$$\mathcal{U}_1 |sa\rangle |0\rangle^{\otimes k_3} = \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} |sas'\rangle$$

For the encoding of the sensor model of the DDN, the quantum operator $\mathcal{U}_2$ is used as in Definition A.4:

Definition A.4. *$\mathcal{U}_2$ is a controlled quantum operator encoding the sensor model $P(O_{t+1}|s_{t+1}, a_t)$ on the qubits of RV O_{t+1} given the values s_{t+1} and a_t of RVs S_{t+1} and A_t , respectively:*

$$\mathcal{U}_2 |as'\rangle |0\rangle^{\otimes k_4} = \sum_{o \in \Omega} \sqrt{P(o|s', a)} |as'o\rangle$$

The final encoding operator is $\mathcal{U}_3$, which encodes the reward dynamics of the DDN into the quantum circuit, as expressed by Definition A.5:

Definition A.5. *The quantum operator $\mathcal{U}_3$ encodes the reward dynamics $P(R_{t+1}|s_t, a_t)$ into the qubits of RV R_{t+1} given the values s_{t+1} and a_t of RVs S_t and A_t , respectively:*

$$\mathcal{U}_3 |sa\rangle |0\rangle^{\otimes k_5} = \sum_{r \in \mathcal{R}} \sqrt{P(r|s, a)} |sar\rangle$$

Applying all these encoding operators, in the order they were presented in the definitions above, is the same as encoding the DDN. They can also be grouped into another quantum operator $\mathcal{B}$, the DDN encoding operator, as in Lemma A.6. This joint operator can be implemented by the encoding scheme discussed in section 3.5; the rotation operators are responsible for encoding the CPTs of a belief update DDN into the quantum circuit.

Lemma A.6. *The DDN encoding operator $\mathcal{B}$ is composed of the sequential application of quantum operators $\mathcal{U}(b)$, $\mathcal{U}(b)$, $\mathcal{U}_1$, $\mathcal{U}_2$ and $\mathcal{U}_3$ fom Definitions A.1 up to A.5, in this order. Its application results in the following quantum state:*

$$\mathcal{B} |0\rangle^{\otimes N} = \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sum_{o \in \Omega} \sqrt{P(o|s', a)} \sum_{r \in \mathcal{R}} \sqrt{P(r|s', a)} |sas'or\rangle$$

Proof. First, the operators $\mathcal{U}(b)$ and $\mathcal{U}(a)$ are applied to the initial quantum state $|0\rangle^{\otimes n}$, according to Definitions A.1 and A.2:

$$|\psi_1(b, a)\rangle = (\mathcal{U}(b) \otimes \mathcal{U}(a) \otimes \mathbb{I}) |0\rangle^{\otimes n} = \sum_{s \in \mathcal{S}} \sqrt{b(s)} |sa\rangle |0\rangle^{\otimes (k_3+k_4+k_5)}$$

Then, the transition dynamics encoding operator $\mathcal{U}_1$ is applied to the quantum state $|\psi_1(b, a)\rangle$ as in Definition A.3:

$$|\psi_2(b, a)\rangle = (\mathcal{U}_1 \otimes \mathbb{I}) |\psi_1(b, a)\rangle \stackrel{(A.3)}{=} \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} |sas'\rangle |0\rangle^{\otimes(k_4+k_5)}$$

Afterwards, the sensor model encoding operator $\mathcal{U}_2$ is applied to $|\psi_2(b, a)\rangle$, according to Definition A.4, yielding:

$$|\psi_3(b, a)\rangle = (\mathbb{I} \otimes \mathcal{U}_2 \otimes \mathbb{I}) |\psi_2(b, a)\rangle \stackrel{(A.4)}{=} \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sum_{o \in \Omega} \sqrt{P(o|s')} |sas'o\rangle |0\rangle^{\otimes k_5}$$

Finally, using Definition A.5, the reward dynamics encoding operator $\mathcal{U}_3$ is applied to $|\psi_3(b, a)\rangle$. The output quantum state is given by:

$$|\psi_4(b, a)\rangle = (\mathbb{I} \otimes \mathcal{U}_3) |\psi_3(b, a)\rangle \stackrel{(A.5)}{=} \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sum_{o \in \Omega} \sqrt{P(o|s')} \sum_{r \in \mathcal{R}} \sqrt{P(r|s', a)} |sas'or\rangle$$

□

The DDN encoding operator is therefore used to create the amplitude amplification operator $G(o) = \mathcal{BS}_0\mathcal{B}^\dagger\mathcal{S}_e$. In this section, it is assumed that the number k always gives a solution state with 100% probability, as it still proves that the quantum circuit of Figure 3 is correct.

Thus, assuming a perfect parameter k for the number of applications of the amplitude amplification operator, this operator will completely zero all the amplitudes of the quantum states with the wrong observation $O_{t+1} \neq o$, amplifying the amplitudes of quantum states with the observation $O_{t+1} = o$, as follows:

Assertion A.7. *Suppose the amplitude amplification operator $G(o)$ is applied to a quantum state $|\psi\rangle = \sum_{x \in \mathcal{X}} \sum_{o' \in \Omega} \alpha_{x,o'} |x, o'\rangle$ (a superposition over values of the sets $\mathcal{X}$ and observations Ω) a perfect number k of times. In this scenario, this operator amplifies the amplitudes of quantum states with observations o , while leaving all other states with zero amplitude:*

$$G^k(o) |\psi\rangle = \frac{1}{\sqrt{\eta}} \sum_{x \in \mathcal{X}} \alpha_{x,o} |x, o\rangle$$

where $\eta = \sum_{x \in \mathcal{X}} |\alpha_{x,o}|^2$ is a normalization constant such that the resulting quantum state $|\psi'\rangle = G^k(o) |\psi\rangle$ obeys the relationship $\langle\psi'|\psi'\rangle = 1$.

The amplitude amplification operator can be implemented using the phase flip operator discussed in Subsection A.2. Now, with every operator defined, Lemma A.8 derives the final quantum state after applying the encoding circuit depicted in Figure 3.

Lemma A.8. *The final quantum state $|\psi_{\text{final}}(b, a, o)\rangle$ of the quantum circuit depicted in Figure 3 is given by the following equation:*

$$|\psi_{\text{final}}(b, a, o)\rangle = \frac{1}{\sqrt{\eta}} \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sqrt{P(o|s', a)} \sum_{r \in \mathcal{R}} \sqrt{P(r|s', a)} |sas'or\rangle$$

where $\eta = \sum_{s' \in \mathcal{S}} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s)$ is a normalization constant.

Proof. The final quantum state is the application of the amplitude amplification operator to the output quantum state of the DDN encoding operator of Lemma A.6:

$$\begin{aligned} |\psi_{\text{final}}(b, a, o)\rangle &= G^k(o) \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sum_{o \in \Omega} \sqrt{P(o|s', a)} \sum_{r \in \mathcal{R}} \sqrt{P(r|s', a)} |sas'or\rangle \\ &\stackrel{(A.7)}{=} \frac{1}{\sqrt{\eta}} \sum_{s \in \mathcal{S}} \sqrt{b(s)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sqrt{P(o|s', a)} \sum_{r \in \mathcal{R}} \sqrt{P(r|s', a)} |sas'or\rangle \end{aligned}$$

□

The quantum state $|\psi_{\text{final}}\rangle$ is the result of the application of every quantum operator in the circuit of Figure 3. The measurement, the final operation of this circuit, is performed only on the qubit of S_{t+1} . The likelihood of an outcome of this measurement can be derived from the partial density matrix $\rho(b, a, o)$ only representing qubit S_{t+1} . In turn, this matrix can be calculated by executing a partial trace (over every other RV except S_{t+1}) in the density matrix $|\psi_{\text{final}}(b, a, o)\rangle \langle \psi_{\text{final}}(b, a, o)|$ representing the final quantum state. Using this approach, Lemma A.9 calculates the measurement probabilities of the quantum circuit of Figure 3:

Lemma A.9. *The probability of measuring $S_{t+1} = s'$ in the quantum circuit of Figure 3 is given by the following expression:*

$$\langle s' | \rho(b, a, o) | s' \rangle = \frac{1}{\eta} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s) \quad (11)$$

Proof. First, let us compute the partial density matrix:

$$\begin{aligned} \rho(b, a, o) &= \sum_{s, a', o', r} \langle sa'o'r | \psi_{\text{final}}(b, a, o) \rangle \langle \psi_{\text{final}}(b, a, o) | sa'o'r \rangle \\ &= \frac{1}{\eta} \sum_{s, a', o', r} \left(\sum_{s_* \in \mathcal{S}} \sqrt{b(s_*)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s_*, a)} \sqrt{P(o|s', a)} \sum_{r' \in \mathcal{R}} \sqrt{P(r'|s_*, a)} \langle sa'o'r | s_* a s' o r' \rangle \right) \\ &\quad \left(\sum_{s_* \in \mathcal{S}} \sqrt{b(s_*)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s_*, a)} \sqrt{P(o|s', a)} \sum_{r' \in \mathcal{R}} \sqrt{P(r'|s_*, a)} \langle s_* a s' o r' | sa'o'r \rangle \right) \\ &= \frac{1}{\eta} \sum_{s, a', o', r} \left(\sum_{s_* \in \mathcal{S}} \sqrt{b(s_*)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s_*, a)} \sqrt{P(o|s', a)} \sum_{r' \in \mathcal{R}} \sqrt{P(r'|s_*, a)} \delta_{ss_*} \delta_{aa'} \delta_{oo'} \delta_{rr'} |s'\rangle \right) \\ &\quad \left(\sum_{s_* \in \mathcal{S}} \sqrt{b(s_*)} \sum_{s' \in \mathcal{S}} \sqrt{P(s'|s_*, a)} \sqrt{P(o|s', a)} \sum_{r' \in \mathcal{R}} \sqrt{P(r'|s_*, a)} \delta_{ss_*} \delta_{aa'} \delta_{oo'} \delta_{rr'} \langle s' | \right) \end{aligned}$$

$$= \frac{1}{\eta} \sum_{s \in \mathcal{S}} b(s) \sum_{r \in \mathcal{R}} \left(\sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sqrt{P(o|s', a)} \sqrt{P(r|s, a)} |s'\rangle \right) \\ \left(\sum_{s' \in \mathcal{S}} \sqrt{P(s'|s, a)} \sqrt{P(o|s', a)} \sqrt{P(r|s, a)} \langle s'| \right)$$

Then, the probability of measuring state S_{t+1} with value s' is computed as follows:

$$\begin{aligned} \langle s' | \rho(b, a, o) | s' \rangle &= \frac{1}{\eta} \sum_{s \in \mathcal{S}} b(s) \sum_{r \in \mathcal{R}} \left(\sum_{s^* \in \mathcal{S}} \sqrt{P(s^*|s, a)} \sqrt{P(o|s^*, a)} \sqrt{P(r|s, a)} \langle s' | s^* \rangle \right) \\ &\quad \left(\sum_{s^* \in \mathcal{S}} \sqrt{P(s^*|s, a)} \sqrt{P(o|s^*, a)} \sqrt{P(r|s, a)} \langle s^* | s' \rangle \right) \\ &= \frac{1}{\eta} \sum_{s \in \mathcal{S}} b(s) \sum_{r \in \mathcal{R}} \left(\sum_{s^* \in \mathcal{S}} \sqrt{P(s^*|s, a)} \sqrt{P(o|s^*, a)} \sqrt{P(r|s, a)} \delta_{s' s^*} \right) \\ &\quad \left(\sum_{s^* \in \mathcal{S}} \sqrt{P(s^*|s, a)} \sqrt{P(o|s^*, a)} \sqrt{P(r|s, a)} \delta_{s^* s'} \right) \\ &= \frac{1}{\eta} \sum_{s \in \mathcal{S}} b(s) \sum_{r \in \mathcal{R}} P(s'|s, a) P(o|s', a) P(r|s, a) \\ &= \frac{1}{\eta} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s) \sum_{r \in \mathcal{R}} P(r|s, a) \\ &= \frac{1}{\eta} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s) \end{aligned}$$

□

Once the probabilities of measurement of the final quantum state of the belief update quantum circuit are known, let us rename $\tau_q(b, a, o)(s') = \langle s' | \rho(b, a, o) | s' \rangle$ as the quantum belief update rule. Thus, Theorem A.10 proves the initial claim, that the quantum and classical belief update rules, as in Equation (A.8), are equivalent:

Theorem A.10. *The quantum and classical belief update rules $\tau_q(b, a, o)$ and $\tau_c(b, a, o)$, for DDNs using rejection sampling, are equivalent:*

$$\tau_q(b, a, o)(s') = \tau_c(b, a, o)(s'), \quad \forall s' \in \mathcal{S}$$

Proof. Equation (11) can be re-written as:

$$\tau_q(b, a, o)(s') = \frac{1}{\eta} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s)$$

It is also known from the classical case that the classical belief update can be re-written to incorporate a proportionality constant:

$$\tau_c(b, a, o)(s') \stackrel{((A.7))}{=} \frac{1}{\eta'} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s)$$

Since both $\tau_q(b, a, o)(s')$ and $\tau_c(b, a, o)(s')$ are probability distributions, their sum over s' is the same, which is equal to one, and therefore:

$$\begin{aligned} \sum_{s'} \tau_q(b, a, o)(s') &= \sum_{s'} \tau_c(b, a, o)(s') \\ \Leftrightarrow \frac{1}{\eta} \sum_{s' \in \mathcal{S}} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s) &= \frac{1}{\eta'} \sum_{s' \in \mathcal{S}} P(o|s', a) \sum_{s \in \mathcal{S}} P(s'|s, a) b(s) \\ \Leftrightarrow \eta &= \eta' \end{aligned}$$

Therefore, the quantum and the classical belief update are equivalent:

$$\tau_q(b, a, o)(s') = \tau_c(b, a, o)(s'), \quad \forall s' \in \mathcal{S}$$

□

As a consequence of Theorem 4.2, the belief updating procedure on any algorithm resorting to it can be performed in a quantum setting, leveraging the quadratic speedup of quantum rejection sampling.

Naturally, the way to perform this quantum belief update operation is not to just run the circuit once and retrieve a measurement. What is shown in Theorem 4.2 is that the probabilities of obtaining a certain measurement match the probabilities of the classical belief update rule. Nonetheless, obtaining an approximation of the next belief state using this circuit should be done with a certain number of samples n .

The simplest way to proceed is to run the quantum circuit of Figure 3 n times (n being the desired number of samples), extracting the samples $s'_i \sim \tau_q(b, a, o)$, $i = \{1, \dots, n\}$ for the next state, and approximating the next belief state $b'(s')$ by the following equation:

$$b'_n(s') = \frac{1}{n} \sum_{i=1}^n \delta_{s' s'_i} \quad (\text{A.9})$$

where b'_n is an approximation of the belief state, computed resorting to n samples. Equation (A.9) simply states that the next belief state value for state s' is the fraction of extracted samples that match state s' .

B. Complexity Analysis

In this Appendix we perform an execution time complexity analysis of both the classical and quantum-classical lookahead algorithms in order to compare the two and check for potential speedups in the quantum approach.

Both of these algorithms are probabilistic. To execute them, samples have to be extracted and used to approximate probability distributions, which are then used to calculate a near-optimal action. As such, to determine their computational complexity, we start by determining their *sample complexity*: the total number of samples needed to execute the algorithm.

This appendix is therefore divided into two sections. B.1 discusses the sample complexity of the lookahead algorithm, giving conditions under which there are formal guarantees for the algorithm to run with an error

of at most ϵ , and a confidence interval of at least $1 - \delta$. This analysis applies to both the classical and quantum algorithms, as the number of samples that need to be extracted is the same in both cases.

These results are then used in B.2 to determine the computational complexity.

B.3 lays out four lemmas used in these analyses, which we present separately for ease of reading.

B.1. Sample Complexity

The sample complexity of the quantum-classical algorithm counts the total number of samples ³ required to achieve a certain goal. In this analysis, the chosen goal is to reach an ϵ approximation of the action value function of the optimal action, with a confidence interval of $1 - \delta$.

To exemplify, consider that the value r is approximated by the output $r^{\mathcal{L}}$ of a probabilistic algorithm $\mathcal{L}$. For a given precision ϵ , the output of a single execution of $\mathcal{L}$ is said to be ϵ -approximate if:

$$|r - r^{\mathcal{L}}| \leq \epsilon \quad (\text{B.1})$$

However, if algorithm $\mathcal{L}$ is probabilistic, the error of the approximation varies from one execution to another. As a consequence, if this algorithm is executed an extremely high number of times, the approximation error of some of these executions is likely to be higher than the defined precision. Therefore, most probabilistic algorithms can not be called ϵ -approximate, as not every execution obeys Equation (B.1).

A better alternative is not to force the approximation of every execution to have a precision of ϵ , but to guarantee instead that each execution has at least some probability $1 - \delta$ (confidence interval) of having an error no greater than ϵ :

$$P(|r - r^{\mathcal{L}}| \leq \epsilon) \geq 1 - \delta \quad (\text{B.2})$$

Consider the definition of the optimal action value function $Q^*(\mathbf{b}_t, a_t)$ ⁴:

$$Q^*(\mathbf{b}_t, a_t) = \mathbb{E}(r_{t+1} | \mathbf{b}_t, a_t) + \gamma \sum_{o_{t+1}} P(o_{t+1} | \mathbf{b}_t, a_t) \max_{a_{t+1} \in \mathcal{A}} Q^*(\mathbf{b}_{t+1}, a_{t+1})$$

$$\mathbf{b}_{t+1}(s_{t+1}) = P(s_{t+1} | o_{t+1}, a_t, \mathbf{b}_t)$$

The analysis of the sample complexity of the algorithm entails many manipulations of these expressions. So, for the sake of compactness and comprehensibility of the proofs of the results ahead, it is useful to resort to vectorial notation ⁵. As such, let $\mathbf{p}_{r_t} \in \Delta_{\mathcal{R}}$ ⁶ represent the probability distribution $P(r_{t+1} | \mathbf{b}_t, a_t), \forall r_{t+1} \in \mathcal{R}$, $\mathbf{p}_{o_t} \in \Delta_{\Omega}$ represent the probability distribution $P(o_{t+1} | \mathbf{b}_t, a_t), \forall o_{t+1} \in \Omega$, $\mathbf{r} \in \mathbb{R}^{|\mathcal{R}|}$

³which corresponds to the total amount of times that the dynamic decision network (DDN) has to be called

⁴Vector quantities are represented in bold.

⁵Although vectorial notation is preferred in this section, explicitly writing out the probability distributions is sometimes clearer and occasionally used.

⁶In this notation, $\mathbf{p} \in \Delta_{\mathcal{V}}$ means that $\mathbf{p}$ is a probability distribution vector over the space $\mathcal{V}$.

be a vector whose entries enumerate every possible reward and $\mathbf{V}_{t+1}^* \in \mathbb{R}^{|\Omega|}$ be a vector whose entries are the values $\max_{a_{t+1} \in \mathcal{A}} Q^*(\mathbf{b}_{t+1}, a_{t+1})$ for every belief update of $\mathbf{b}_t$ after taking action a_t ⁷. The previous equation can be re-written, using this notation, as⁸:

$$Q^*(\mathbf{b}_t, a_t) = \mathbf{p}_{r_t}^\top \mathbf{r} + \gamma \mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^* \quad (\text{B.3})$$

The lookahead algorithm $\mathcal{L}$ calculates a near-optimal value function $Q^\mathcal{L}$ similarly to that of Equation (B.3), but replacing $\mathbf{p}_{r_t}$ with its empirical estimation $\mathbf{p}_{r_t, n_t}$ using n_t samples, $\mathbf{p}_{o_t}$ with its empirical estimation $\mathbf{p}_{o_t, m_t}$ using m_t samples and $\mathbf{V}_{t+1}^*$ with its empirical estimation $\mathbf{V}_{t+1}^\mathcal{L}$ calculated recursively. Thus, the lookahead algorithm, starting with root belief node $\mathbf{b}_t$ at time-step t obeys the following recursive Equation (B.4) with a terminal case described by Equation (B.5) (according to the lookahead horizon H):

$$Q^\mathcal{L}(\mathbf{b}_t, a_t) = \mathbf{p}_{r_t, n_t}^\top \mathbf{r} + \gamma \mathbf{p}_{o_t, m_t}^\top \mathbf{V}_{t+1}^\mathcal{L} \quad (\text{B.4})$$

$$Q^\mathcal{L}(\mathbf{b}_{t+H}, a_{t+H}) = 0 \quad (\text{B.5})$$

So far, three sources of error for the lookahead algorithm have been mentioned:

- Approximating the reward distribution $\mathbf{p}_{r_t}$ by its empirical estimation $\mathbf{p}_{r_t, n_t}$, using n_t samples.
- Approximating the observation distribution $\mathbf{p}_{o_t}$ by its empirical estimation $\mathbf{p}_{o_t, m_t}$, using m_t samples.
- The stop case for the lookahead algorithms in the horizon H limit.

There is yet another source of error, coming from the estimation of the belief state. This estimation, however, differs slightly from the estimations of the two distributions mentioned above. Suppose we have an empirical estimation $\mathbf{b}_{l_t}$ of the belief state $\mathbf{b}_t$, using l_t samples. When performing the belief update to get the next belief state $\mathbf{b}_{l_{t+1}}(s_{t+1}) = P_{l_{t+1}}(s_{t+1}|o_{t+1}, a_t, \mathbf{b}_{l_t})$, this quantity estimates $P(s_{t+1}|o_{t+1}, a_t, \mathbf{b}_{l_t})$ rather than the belief-update $P(s_{t+1}|o_{t+1}, a_t, \mathbf{b}_t)$ of the true belief-state $\mathbf{b}_t$. As a consequence, $\mathbf{b}_{l_{t+1}}$ has two sources of error: the fact that the belief update is performed on a previous estimator and the sampling process that comes after it.

Definition B.1 introduces the lookahead error using the notion of *regret* Liu et al. (2021), a scalar quantity generally used in RL to define an algorithm's error. It is defined as the absolute difference between the utility of the best action that could be taken and the utility of the action that the agent actually took. The remainder of this section serves to provide and analyze an upper bound to this error in order to give theoretical guarantees concerning the performance of the algorithm. These theoretical guarantees are the conditions under which the algorithm can be considered ϵ -approximate with a confidence interval of $1 - \delta$, as per Equation (B.2).

⁷Since the belief update rule is $\mathbf{b}_{t+1}(s_{t+1}) = P(s_{t+1}|o_{t+1}, a_t, \mathbf{b}_t)$ and both a_t and $\mathbf{b}_t$ are fixed, there are $|\Omega|$ different possible $\mathbf{b}_{t+1}$, one for each observation $o_{t+1} \in \Omega$.

⁸In vector notation, $\mathbf{v}^\top$ is the transpose of $\mathbf{v}$, such that the inner-product of two vectors $\mathbf{u}$ and $\mathbf{v}$ can be written as $\mathbf{u}^\top \mathbf{v}$.

Definition B.1. (*Lookahead error*). Let $a_t^{\mathcal{L}} = \operatorname{argmax}_{a_t \in \mathcal{A}} Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t)$ be the near optimal action chosen by the algorithm at time t . The lookahead error ϵ_t at time t is defined as the regret:

$$\epsilon_t = \left| \max_{a_t \in \mathcal{A}} Q^*(\mathbf{b}_t, a_t) - Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) \right| \quad (\text{B.6})$$

Using Lemmas B.20 and B.21, Lemma B.2 decomposes the lookahead error into three distinct errors, each one easier to bound than the original one:

Lemma B.2. *The lookahead error ϵ_t can be bounded by the following expression:*

$$\epsilon_t \leq \max_{a_t \in \mathcal{A}} (|Q^*(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_t, a_t)| + |Q^{\mathcal{L}}(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t)|) + |Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t^{\mathcal{L}})|$$

Proof.

$$\begin{aligned} \epsilon_t &\stackrel{(B.1)}{=} \left| \max_{a_t \in \mathcal{A}} Q^*(\mathbf{b}_t, a_t) - Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) \right| \\ &\stackrel{(B.21)}{\leq} \left| \max_{a_t \in \mathcal{A}} Q^*(\mathbf{b}_t, a_t) - \max_{a_t \in \mathcal{A}} Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t) \right| + \left| Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) - \max_{a_t \in \mathcal{A}} Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t) \right| \\ &\stackrel{(B.20)}{\leq} \max_{a_t \in \mathcal{A}} |Q^*(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t)| + |Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t^{\mathcal{L}})| \\ &= \max_{a_t \in \mathcal{A}} (|Q^*(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_t, a_t)| + |Q^{\mathcal{L}}(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t)|) + |Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t^{\mathcal{L}})| \\ &\stackrel{(B.21)}{\leq} \max_{a_t \in \mathcal{A}} (|Q^*(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_t, a_t)| + |Q^{\mathcal{L}}(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t)|) + |Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t^{\mathcal{L}})| \end{aligned}$$

□

Lemma B.2 states that the total lookahead error can be decomposed into three distinct errors:

- $|Q^*(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_t, a_t)|$: the error between the true action value function and the approximate one, both evaluated at belief state $\mathbf{b}_t$.
- $|Q^{\mathcal{L}}(\mathbf{b}_t, a_t) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t)|$: the error between the approximate action value function evaluated at the true belief state $\mathbf{b}_t$ and at the approximate belief-state $\mathbf{b}_{l_t}$.
- $|Q^*(\mathbf{b}_{l_t}, a_t^{\mathcal{L}}) - Q^{\mathcal{L}}(\mathbf{b}_{l_t}, a_t^{\mathcal{L}})|$: the error between the true and approximate action value functions, both evaluated at the approximate belief $\mathbf{b}_{l_t}$ and the near-optimal action a_t^* .

The remainder of this section determines upper bounds to all error terms in order to define an upper bound for the lookahead error. The main result that allows defining these bounds is Hoeffding's inequality, stated in Theorem B.3⁹.

⁹ $\|\cdot\|_{\infty}$ denotes the l_{∞} norm of a vector, which extracts its maximum element. If $u = \|\mathbf{U}\|_{\infty}$, then $u \geq u', \forall u' \in \mathbf{U}$.

Theorem B.3. (Hoeffding's Inequality Sidford et al. (2018)). Let $\mathbf{p} \in \Delta_{\mathcal{V}}$ be a probability vector and $\mathbf{V} \in \mathbb{R}^{|\mathcal{V}|}$ a vector. Let $\mathbf{p}_m \in \Delta_{\mathcal{V}}$ be an empirical estimation of $\mathbf{p}$ using m i.i.d.¹⁰ samples and $\delta \in (0, 1)$ a parameter. Then, with probability at least $1 - \delta$:

$$|\mathbf{p}^\top \mathbf{V} - \mathbf{p}_m^\top \mathbf{V}| \leq \|\mathbf{V}\|_\infty \mathcal{H}(m, \delta)$$

Where $\mathcal{H}(m, \delta) = \sqrt{\frac{1}{2m} \log \left(\frac{2}{\delta} \right)}$.

The results above already provide the necessary tools to bound two of the error sources that compose the lookahead error as in Lemma B.2. Using these results, Lemma B.4 bounds the error between the true value function and the approximate one, both evaluated at the same belief state and action:

Lemma B.4. Let $\Gamma = (1 - \gamma)^{-1}$ be the effective horizon, $r_{\max} = \|\mathbf{r}\|_\infty$ be the maximum reward and $\sigma_t \in (0, 1)$ a parameter defining the confidence interval $1 - \sigma_t$ for sampling both $\mathbf{p}_{r_t}$ and $\mathbf{p}_{o_t}$ at time-step t . The error $|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)|$ has the following upper bound:

$$|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| \leq r_{\max} \sum_{k=0}^{H-1} \gamma^k (\mathcal{H}(n_{t+k}, \sigma_{t+k}) + \gamma \Gamma \mathcal{H}(m_{t+k}, \sigma_{t+k})) + \gamma^H \Gamma r_{\max}$$

Proof. Using the definitions for Q^* and $Q^\mathcal{L}$ from Equations (B.3) and (B.4), respectively:

$$|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| \stackrel{(B.21)}{\leq} |\mathbf{r}^\top \mathbf{p}_{r_t} - \mathbf{r}^\top \mathbf{p}_{r_t, n_t}| + \gamma |\mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^* - \mathbf{p}_{o_t, m_t}^\top \mathbf{V}_{t+1}^\mathcal{L}|$$

Identity $\mathbf{p}_{o_t} - \mathbf{p}_{o_t, m_t} \leq |\mathbf{p}_{o_t} - \mathbf{p}_{o_t, m_t}| \Leftrightarrow -\mathbf{p}_{o_t, m_t} \leq |\mathbf{p}_{o_t} - \mathbf{p}_{o_t, m_t}| - \mathbf{p}_{o_t}$ yields:

$$|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| \stackrel{(B.21)}{\leq} |\mathbf{r}^\top \mathbf{p}_{r_t} - \mathbf{r}^\top \mathbf{p}_{r_t, n_t}| + \gamma |\mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^\mathcal{L} - \mathbf{p}_{o_t, m_t}^\top \mathbf{V}_{t+1}^\mathcal{L}| + \gamma |\mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^* - \mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^\mathcal{L}|$$

The terms $|\mathbf{r}^\top \mathbf{p}_{r_t} - \mathbf{r}^\top \mathbf{p}_{r_t, n_t}|$ and $|\mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^\mathcal{L} - \mathbf{p}_{o_t, m_t}^\top \mathbf{V}_{t+1}^\mathcal{L}|$ can be bounded through Hoeffding's inequality in Theorem B.3 and the fact that, by definition, $\|\mathbf{V}_{t+1}^\mathcal{L}\|_\infty \leq Q_{\max} = \Gamma r_{\max}$. The other term can be decomposed into a sum:

$$\begin{aligned} |Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| &\stackrel{(B.3)}{\leq} r_{\max} (\mathcal{H}(n_t, \sigma_t) + \gamma \Gamma \mathcal{H}(m_t, \sigma_t)) \\ &\quad + \gamma \max_{a_{t+1} \in \mathcal{A}} \sum_{o_{t+1} \in \Omega} P(o_{t+1} | \mathbf{b}_t, a_t) |Q^*(\mathbf{b}_{t+1}, a_{t+1}) - Q^\mathcal{L}(\mathbf{b}_{t+1}, a_{t+1})| \end{aligned}$$

Let μ_{t+1} be a belief-state and a'_{t+1} an action such that

$$|Q^*(\mu_{t+1}, a'_{t+1}) - Q^\mathcal{L}(\mu_{t+1}, a'_{t+1})| = \max_{a_{t+1}, \mathbf{b}_{t+1}} |Q^*(\mathbf{b}_{t+1}, a_{t+1}) - Q^\mathcal{L}(\mathbf{b}_{t+1}, a_{t+1})|$$

¹⁰Independent and identically distributed

Then:

$$\begin{aligned}
|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| &\leq r_{\max} (\mathcal{H}(n_t, \sigma_t) + \gamma \Gamma \mathcal{H}(m_t, \sigma_t)) \\
&\quad + \gamma |Q^*(\mu_{t+1}, a'_{t+1}) - Q^\mathcal{L}(\mu_{t+1}, a'_{t+1})| \sum_{o_{t+1} \in \Omega} P(o_{t+1} | \mathbf{b}_t, a_t) \\
&= r_{\max} (\mathcal{H}(n_t, \sigma_t) + \gamma \Gamma \mathcal{H}(m_t, \sigma_t)) \\
&\quad + \gamma |Q^*(\mu_{t+1}, a'_{t+1}) - Q^\mathcal{L}(\mu_{t+1}, a'_{t+1})|
\end{aligned}$$

Finally, with the recursive application of this inequality to $|Q^*(\mu_{t+1}, a'_{t+1}) - Q^\mathcal{L}(\mu_{t+1}, a'_{t+1})|$, with Equation (B.5) as a stop case to the approximate action value function, the result of this lemma is attained:

$$\begin{aligned}
|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| &\leq r_{\max} \sum_{k=0}^{H-1} \gamma^k (\mathcal{H}(n_{t+k}, \sigma_{t+k}) + \gamma \Gamma \mathcal{H}(m_{t+k}, \sigma_{t+k})) \\
&\quad + \gamma^H |Q^*(\mu_{t+H}, a'_{t+H}) - Q^\mathcal{L}(\mu_{t+H}, a'_{t+H})| \\
&\stackrel{((B.5))}{=} r_{\max} \sum_{k=0}^{H-1} \gamma^k (\mathcal{H}(n_{t+k}, \sigma_{t+k}) + \gamma \Gamma \mathcal{H}(m_{t+k}, \sigma_{t+k})) \\
&\quad + \gamma^H |Q^*(\mu_{t+H}, a_{t+H}^*)| \\
&\leq r_{\max} \sum_{k=0}^{H-1} \gamma^k (\mathcal{H}(n_{t+k}, \sigma_{t+k}) + \gamma \Gamma \mathcal{H}(m_{t+k}, \sigma_{t+k})) + \gamma^H \Gamma r_{\max}
\end{aligned}$$

□

Lemma B.4 gives a bound to the difference between the true value function and the value function calculated by the lookahead algorithm, both evaluated at the same action and belief state. Notice, however, that the bound holds for any choice of action and belief state, because it is independent of both these quantities. Thus, both $|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)|$ and $|Q^*(\mathbf{b}_{l_t}, a_t^\mathcal{L}) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t^\mathcal{L})|$ can be bounded by this lemma.

Before moving on to the next lemma, let us first count the total number of Hoeffding inequalities that were used in the proof of Lemma B.4. This count will be useful later on to calculate the confidence interval of the lookahead algorithm's bounds. The bound in Lemma B.4 was attained by splitting the error source $|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)|$ into two terms, one where the Hoeffding bounds are applied ($|\mathbf{r}^\top \mathbf{p}_{r_t} - \mathbf{r}^\top \mathbf{p}_{r_t, n_t}| + \gamma |\mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^\mathcal{L} - \mathbf{p}_{o_t, m_t}^\top \mathbf{V}_{t+1}^\mathcal{L}|$) and a recursive term ($\gamma |\mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^* - \mathbf{p}_{o_t}^\top \mathbf{V}_{t+1}^\mathcal{L}|$) where the same procedure is applied until a stop case is found. Let's call these two terms *immediate* and *recursive*, respectively. To bound the immediate term, 2 Hoeffding inequalities were used. For the first time-step in the recursive term, $2A\Omega$ Hoeffding inequalities are needed (2 Hoeffding inequalities as per the immediate term, but for every action and every observation). As such, a total of $2 \sum_{k=0}^{H-1} A^k \Omega^k$ Hoeffding inequalities are used to derive the bound in Lemma B.4.

To bound the full lookahead error, only the error of approximating the belief state remains to be bounded. As mentioned above, the belief states for the lookahead algorithm are approximated in a different manner from the reward and observation distributions. The only true belief state that is known to the lookahead

algorithm is the prior $\mathbf{b}_0$, and therefore, the belief estimations $\mathbf{b}_{l_t}$ are calculated by performing a belief update on the previous estimation $\mathbf{b}_{l_{t-1}}$, yielding the distribution $\mathbf{p}_t \in \Delta_{\mathcal{S}}$ with entries $P(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}})$, and extracting l_t samples from it, creating an empirical estimation distribution $\mathbf{p}_{t,l_t} = \mathbf{b}_{l_t} \in \Delta_{\mathcal{S}}$ with entries $P_{l_t}(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}})$. As such, Hoeffding's inequality can not be applied to the distributions $\mathbf{b}_t$ and $\mathbf{b}_{l_t}$, because they are not direct estimations of one another. It can be applied, however, to $\mathbf{p}_t$ and $\mathbf{p}_{t,l_t}$, since the latter approximates the former distribution via sampling, as stated above.

Lemma B.5 uses an indirect approach to applying the Hoeffding inequalities and still be able to bound the error between the true belief state $\mathbf{b}_t$ and the approximate belief state $\mathbf{b}_{l_t}$ of the algorithm at time t :

Lemma B.5. *Given a distribution $\mathbf{p}_{s_t}$ from a family of distributions $P(\cdot|s_t, \cdot)$ conditioned on the state $s_t \in \mathcal{S}$ ¹¹, the following inequality bounds the error of approximating the belief-state at time t , using l_k samples and a confidence interval of $1 - \sigma_k$ to estimate it at time-step k :*

$$|\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| \leq \mathcal{S}^t \sum_{k=0}^t \frac{1}{\mathcal{S}^k} \mathcal{H}(l_k, \sigma_k)$$

Proof.

$$|\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| = \left| \sum_{s_t \in \mathcal{S}} P(\cdot|s_t, \cdot) (P(s_t|o_t, a_{t-1}, \mathbf{b}_{t-1}) - P_{l_t}(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}})) \right|$$

Notice that the difference $P(s_t|o_t, a_{t-1}, \mathbf{b}_{t-1}) - P_{l_t}(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}})$ in the equation above is between two distributions conditioned on different belief states. As such, Hoeffding's inequality can't be directly applied. To overcome this issue, the triangle inequality

$$\begin{aligned} P(s_t|o_t, a_{t-1}, \mathbf{b}_{t-1}) &\leq P(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}}) + |P(s_t|o_t, a_{t-1}, \mathbf{b}_{t-1}) - P(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}})| \\ &= P(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}}) + |\mathbf{p}_{s_{t-1}}^\top \mathbf{b}_{t-1} - \mathbf{p}_{s_{t-1}}^\top \mathbf{b}_{l_{t-1}}| \end{aligned}$$

is used, allowing the expression above to be re-written as:

$$\begin{aligned} |\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| &\leq \sum_{s_t \in \mathcal{S}} P(\cdot|s_t, \cdot) \left(P(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}}) - P_{l_t}(s_t|o_t, a_{t-1}, \mathbf{b}_{l_{t-1}}) + |\mathbf{p}_{s_{t-1}}^\top \mathbf{b}_{t-1} - \mathbf{p}_{s_{t-1}}^\top \mathbf{b}_{l_{t-1}}| \right) \\ &\leq |\mathbf{p}_{s_t}^\top \mathbf{p}_t - \mathbf{p}_{s_t}^\top \mathbf{p}_{t,l_t}| + \mathcal{S} |\mathbf{p}_{s_{t-1}}^\top \mathbf{b}_{t-1} - \mathbf{p}_{s_{t-1}}^\top \mathbf{b}_{l_{t-1}}| \end{aligned}$$

Recursively applying the inequality above and using Hoeffding's inequality:

$$\begin{aligned} |\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| &\leq \sum_{k=0}^t \mathcal{S}^{t-k} |\mathbf{p}_{s_k}^\top \mathbf{p}_k - \mathbf{p}_{s_k}^\top \mathbf{p}_{k,l_k}| \\ &\stackrel{(B.3)}{\leq} \mathcal{S}^t \sum_{k=0}^t \frac{1}{\mathcal{S}^k} \mathcal{H}(l_k, \sigma_k) \end{aligned}$$

□

¹¹In the results of this lemma and the lemmas and theorems that follow, the notation $\mathcal{X}$ for a set $\mathcal{X}$ can have two meanings, that depend on the context of use: it can refer to the set itself, or to the size of the set $|\mathcal{X}|$.

This lemma states that the belief-approximation error bound is cumulative over time. This is intuitive, given that the approximation of a belief state at time-step t is attained by estimating over the previous estimation. Therefore, estimation errors pile up as time progresses.

A few more bounds are useful to derive the full upper bound of the belief error. Due to the fact that an inaccurate belief state is used throughout every time step of the algorithm, not only does this cause errors in the algorithm's belief states themselves, but also in calculating other probability distributions that are conditioned on the belief state (because they should be conditioned on the true belief-state rather than an estimation of it). This implies that both reward sampling and observation sampling also suffer from this belief-state approximation error, and so these error contributions should be accounted for. Lemma B.6 bounds the reward sampling error caused by this belief-approximation:

Lemma B.6. *Let $\mathbf{p} \in \Delta_{\mathcal{R}}$ represent the probability distribution $P(r_{t+1}|\mathbf{b}_t)$ and $\hat{\mathbf{p}} \in \Delta_{\mathcal{R}}$ represent distribution $P(r_{t+1}|\mathbf{b}_t)$. Also, let $\mathbf{p}_{n_t}$ and $\hat{\mathbf{p}}_{n_t}$ be their respective estimations using n_t samples. Then, the following inequality holds:*

$$|\mathbf{p}_{n_t}^\top \mathbf{r} - \hat{\mathbf{p}}_{n_t}^\top \mathbf{r}| \leq r_{\max} \left(2\mathcal{H}(n_t, \sigma_t) + \mathcal{R} \mathcal{S}^t \sum_{k=0}^t \frac{1}{\mathcal{S}^k} \mathcal{H}(l_k, \sigma_k) \right)$$

Proof.

$$\begin{aligned} |\mathbf{p}_{n_t}^\top \mathbf{r} - \hat{\mathbf{p}}_{n_t}^\top \mathbf{r}| &\leq |(\mathbf{p}^\top + |\mathbf{p}^\top - \mathbf{p}_{n_t}^\top|) \mathbf{r} + (-\hat{\mathbf{p}}^\top + |\hat{\mathbf{p}}^\top - \hat{\mathbf{p}}_{n_t}^\top|) \mathbf{r}| \\ &\stackrel{(B.21)}{\leq} |\mathbf{p}^\top \mathbf{r} - \mathbf{p}_{n_t}^\top \mathbf{r}| + |\hat{\mathbf{p}}^\top \mathbf{r} - \hat{\mathbf{p}}_{n_t}^\top \mathbf{r}| + |\mathbf{p}^\top \mathbf{r} - \hat{\mathbf{p}}^\top \mathbf{r}| \\ &\stackrel{(B.3)}{\leq} 2r_{\max} \mathcal{H}(n_t, \sigma_t) + \sum_{r_t \in \mathcal{R}} r_t |\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| \\ &\stackrel{(B.5)}{\leq} r_{\max} \left(2\mathcal{H}(n_t, \sigma_t) + \mathcal{R} \mathcal{S}^t \sum_{k=0}^t \frac{1}{\mathcal{S}^k} \mathcal{H}(l_k, \sigma_k) \right) \end{aligned}$$

□

Analogously to the Lemma B.6, Lemma B.7 bounds the observation sampling error due to the belief-state approximation:

Lemma B.7. *Let $\mathbf{p}_{o_t} \in \Delta_{\Omega}$ be a probability distribution $P(o_{t+1}|\mathbf{b}_t, a_t)$ and $\hat{\mathbf{p}}_{o_t} \in \Delta_{\Omega}$ be $P(o_{t+1}|\mathbf{b}_{l_t}, a_t)$, such that $\mathbf{p}_{o_t, m_t}$ and $\hat{\mathbf{p}}_{o_t, m_t}$ are their empirical estimations with m_t samples. Also, let $\mathbf{V} \in \mathbb{R}^{|\Omega|}$ be a vector of value functions with entries $V(\mathbf{b}_t)$ and $\mathbf{V}^{\mathcal{L}}$ a similar vector with entries $V^{\mathcal{L}}(\mathbf{b}_{l_t})$. Then, the following inequality holds:*

$$|\mathbf{p}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^{\mathcal{L}}| \leq \Gamma r_{\max} \left(2\mathcal{H}(m_t, \sigma_t) + \Omega \mathcal{S}^t \sum_{k=0}^t \frac{1}{\mathcal{S}^k} \mathcal{H}(l_k, \sigma_k) \right) + |\hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^{\mathcal{L}}|$$

Proof.

$$\begin{aligned}
|\mathbf{p}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^\mathcal{L}| &\leq |(\mathbf{p}_{o_t}^\top + |\mathbf{p}_{o_t}^\top - \mathbf{p}_{o_t, m_t}^\top|) \mathbf{V} + (-\hat{\mathbf{p}}_{o_t}^\top + |\hat{\mathbf{p}}_{o_t}^\top - \hat{\mathbf{p}}_{o_t, m_t}^\top|) \mathbf{V}^\mathcal{L}| \\
&\stackrel{(B.21)}{\leq} |\mathbf{p}_{o_t}^\top \mathbf{V} - \mathbf{p}_{o_t, m_t}^\top \mathbf{V}| + |\hat{\mathbf{p}}_{o_t}^\top \mathbf{V}^\mathcal{L} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^\mathcal{L}| + |\mathbf{p}_{o_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t}^\top \mathbf{V}^\mathcal{L}| \\
&\stackrel{(B.3)}{\leq} 2\Gamma r_{\max} \mathcal{H}(m_t, \sigma_t) + |\mathbf{p}_{o_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t}^\top \mathbf{V}| + |\hat{\mathbf{p}}_{o_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t}^\top \mathbf{V}^\mathcal{L}| \\
&= 2\Gamma r_{\max} \mathcal{H}(m_t, \sigma_t) + \sum_{o_{t+1} \in \Omega} V^\mathcal{L}(\mathbf{b}_{t+1}) |\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| + |\hat{\mathbf{p}}_{o_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t}^\top \mathbf{V}^\mathcal{L}| \\
&\leq 2\Gamma r_{\max} \mathcal{H}(m_t, \sigma_t) + \Gamma r_{\max} \Omega |\mathbf{p}_{s_t}^\top \mathbf{b}_t - \mathbf{p}_{s_t}^\top \mathbf{b}_{l_t}| + |\hat{\mathbf{p}}_{o_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t}^\top \mathbf{V}^\mathcal{L}| \\
&\stackrel{(B.5)}{\leq} \Gamma r_{\max} \left(2\mathcal{H}(m_t, \sigma_t) + \Omega \mathcal{S}^t \sum_{k=0}^t \frac{1}{\mathcal{S}^k} \mathcal{H}(l_k, \sigma_k) \right) + |\hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^\mathcal{L}|
\end{aligned}$$

The final recursive term $|\hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^\mathcal{L}|$ is expanded and used to give an upper bound to the full belief error in Lemma B.8. $\square$

Finally, using the results of Lemmas B.6 and B.7, Lemma B.8 provides an upper bound to the belief approximation error:

Lemma B.8. *The belief error $|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)|$ is bounded by the following expression ¹²:*

$$|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)| \leq r_{\max} \sum_{k=0}^{H-1} \gamma^k \left(2\mathcal{H}(n_{t+k}, \sigma_{t+k}) + 2\gamma\Gamma \mathcal{H}(m_{t+k}, \sigma_{t+k}) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^{t+k} \sum_{j=0}^{t+k} \frac{1}{\mathcal{S}^j} \mathcal{H}(l_j, \sigma_j) \right)$$

Proof.

$$|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)| \leq |\mathbf{p}_{n_t}^\top \mathbf{r} - \hat{\mathbf{p}}_{n_t}^\top \mathbf{r}| + \gamma |\mathbf{p}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^\mathcal{L}|$$

using Lemmas B.6 and B.7:

$$\begin{aligned}
|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)| &\leq r_{\max} \left(2\mathcal{H}(n_t, \sigma_t) + 2\gamma\Gamma \mathcal{H}(m_t, \sigma_t) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^t \sum_{j=0}^t \frac{1}{\mathcal{S}^j} \mathcal{H}(l_j, \sigma_j) \right) \\
&\quad + \gamma |\hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V} - \hat{\mathbf{p}}_{o_t, m_t}^\top \mathbf{V}^\mathcal{L}| \\
&\leq r_{\max} \left(2\mathcal{H}(n_t, \sigma_t) + 2\gamma\Gamma \mathcal{H}(m_t, \sigma_t) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^t \sum_{j=0}^t \frac{1}{\mathcal{S}^j} \mathcal{H}(l_j, \sigma_j) \right) \\
&\quad + \gamma \sum_{o_{t+1} \in \Omega} P_{m_t}(o_{t+1} | \mathbf{b}_{l_t}, a_t) \max_{a_{t+1}} |Q^\mathcal{L}(\mathbf{b}_{t+1}, a_{t+1}) - Q^\mathcal{L}(\mathbf{b}_{l_{t+1}}, a_{t+1})|
\end{aligned}$$

¹²In this bound, and throughout the remaining of this section, it is assumed that every sampling operation on the same time-step t (be it reward, observation or belief-state sampling) has the same confidence interval $1 - \sigma_t$.

let μ_{t+1} and a'_{t+1} be a belief-state and an action, respectively, such that

$|Q^\mathcal{L}(\mu_{t+1}, a'_{t+1}) - Q^\mathcal{L}(\mu_{l_{t+1}}, a'_{t+1})| \geq |Q^\mathcal{L}(\mathbf{b}_{t+1}, a_{t+1}) - Q^\mathcal{L}(\mathbf{b}_{l_{t+1}}, a_{t+1})|, \forall \mathbf{b}_{t+1}, a_{t+1}$. Then:

$$|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)| \leq \gamma |Q^\mathcal{L}(\mu_{t+1}, a'_{t+1}) - Q^\mathcal{L}(\mu_{l_{t+1}}, a'_{t+1})| \\ + r_{\max} \left(2\mathcal{H}(n_t, \sigma_t) + 2\gamma\Gamma\mathcal{H}(m_t, \sigma_t) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^t \sum_{j=0}^t \frac{1}{\mathcal{S}^j} \mathcal{H}(l_j, \sigma_j) \right)$$

Using the inequality above recursively, and the stop case from Equation (B.5):

$$|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)| \leq r_{\max} \sum_{k=0}^{H-1} \gamma^k \left(2\mathcal{H}(n_{t+k}, \sigma_{t+k}) + 2\gamma\Gamma\mathcal{H}(m_{t+k}, \sigma_{t+k}) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^{t+k} \sum_{j=0}^{t+k} \frac{1}{\mathcal{S}^j} \mathcal{H}(l_j, \sigma_j) \right)$$

□

Once again, let us count the number of Hoeffding inequalities used to prove this result. Analogously to the count done for Lemma B.4, consider the *immediate* and *recursive* components. Notice that, for the immediate term, $(t+1) + 2$ Hoeffding inequalities are used. The first time-step in the recursive term needs $\mathcal{A}\Omega((t+2) + 2)$ Hoeffding inequalities (it should hold for every action and observation, and the $t+1$ Hoeffding inequalities from the belief-error bound now become $t+2$ because the time-step has increased by one). In total, $\sum_{k=0}^{H-1} \mathcal{A}^k \Omega^k (t+k+2)$ Hoeffding inequalities are needed to prove this bound.

Finally, with the two error terms of the lookahead already bounded, the full lookahead error can be derived:

Theorem B.9. *The lookahead error ϵ_t has the following upper bound:*

$$\epsilon_t \leq r_{\max} \sum_{k=0}^{H-1} \gamma^k \left(4\mathcal{H}(n_{t+k}, \sigma_{t+k}) + 4\gamma\Gamma\mathcal{H}(m_{t+k}, \sigma_{t+k}) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^{t+k} \sum_{j=0}^{t+k} \frac{1}{\mathcal{S}^j} \mathcal{H}(l_j, \sigma_j) \right) \\ + 2\gamma^H \Gamma r_{\max}$$

Proof. Lemma B.2 yields:

$$\epsilon_t \leq \max_{a_t \in \mathcal{A}} (|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| + |Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)|) + |Q^*(\mathbf{b}_{l_t}, a_t^\mathcal{L}) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t^\mathcal{L})|$$

Applying Lemmas B.4 and B.8 to the previous expression concludes the proof. □

The error bound estimated in Theorem B.9 encompasses all four errors of the quantum-classical lookahead algorithm. They can be easily identified from the expression in Theorem B.9:

- *Approximating the reward distribution:* this error term is captured by the n_{t+k} term in the sum.

- *Approximating the observation distribution*: this error term is captured by the m_{t+k} term in the sum.
- *Approximating the belief states*: this error is captured by the l_j term of the bound and increases over time due to repeated approximation of the belief state.
- *The finite horizon of the lookahead*: the lookahead tree has a fixed depth, given by the finite horizon H , introducing a systematic error that can't be compensated by increasing the number of samples. It is captured by the $2\gamma^H \Gamma R_{\max}$ term in the bound.

Theorem B.9 provides an upper bound to the lookahead error of Definition B.1. This upper bound takes into account the use of multiple sampling operations throughout the lookahead tree: reward sampling, observation sampling, and belief-state sampling. Given that the bounds used for each sampling operator fail with a probability of σ_t , the bound for the whole lookahead error also has a non-zero chance of failure, which depends on the confidence interval of each sampling operation and is yet to be defined. This is because the confidence interval $1 - \sigma_t$ only holds locally for each sampling operation, and not for the whole expression presented in Theorem B.9. Naturally, if none of the sampling operation bounds fail, the lookahead error bound holds. As such, the probability of failure δ_t of the lookahead error bound is at least as large as the probability that any one of the sampling bounds fails.

One way to relate the confidence interval $1 - \delta_t$ of the bound in Theorem B.9 to the confidence interval $1 - \sigma_t$ of each sampling operation is by using Theorem B.10. It states that the probability of the union of multiple events is at least as small as the sum of the probabilities of each individual event:

Theorem B.10. (*Boole's Inequality Seneta (1992)*). *Let $Z_1, Z_2, \dots, Z_n$ be probabilistic events that occur with probability $P(Z_1), P(Z_2), \dots, P(Z_n)$. Then:*

$$P\left(\bigcup_{i=1}^n Z_i\right) \leq \sum_{i=1}^n P(Z_i)$$

In the formulation of Theorem B.10, the probabilistic events Z_i could have a higher sampling error than the one provided in Theorem B.3. By attributing this meaning to the probabilistic events, Boole's inequality can be used to relate the sampling bound's confidence interval $1 - \sigma_t$ to the confidence interval of the algorithm's upper bound presented in Theorem B.9.

Lemma B.11. *The confidence interval $1 - \delta_t$ of the algorithm's upper bound given in Theorem B.9 and the confidence interval $1 - \sigma_t$ of the Hoeffding's bound for each sampling operation obeys the following inequality:*

$$\delta_t \leq \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (2 + \mathcal{A}(t + i + 4)) \sigma_{t+i}$$

Proof. Recall, from Lemma B.2, that the lookahead error is bounded by the following expression:

$$\epsilon_t \leq \max_{a_t \in \mathcal{A}} (|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)| + |Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)|) + |Q^*(\mathbf{b}_{l_t}, a_t^\mathcal{L}) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t^\mathcal{L})|$$

The terms $|Q^*(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_t, a_t)|$ and $|Q^*(\mathbf{b}_{l_t}, a_t^\mathcal{L}) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t^\mathcal{L})|$ can both be bounded using Lemma B.4, which needs $2 \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i$ Hoeffding inequalities to be derived. The term $|Q^\mathcal{L}(\mathbf{b}_t, a_t) - Q^\mathcal{L}(\mathbf{b}_{l_t}, a_t)|$ can be bounded using Lemma B.8, which requires $\sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (t + i + 2)$ Hoeffding inequalities. However, two of the terms are under a $\max_{a_t \in \mathcal{A}}$ operation in the lookahead error bound above, so their number of Hoeffding inequalities should be multiplied by a factor $\mathcal{A}$, since these bounds should hold for any possible action. As such, the total number of Hoeffding inequalities to bound the lookahead error is given by:

$$2 \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i + 2\mathcal{A} \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i + \mathcal{A} \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (t + i + 2) = \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (2 + \mathcal{A}(t + i + 4))$$

Each term of the previous sum represents the total number of Hoeffding inequalities at that particular depth in the lookahead tree (if $i = 2$, the corresponding term represents the number of Hoeffding inequalities at depth 2). Given that, for a depth i at time-step t , the probability of a sampling operation failing is σ_{t+i} , Boole's inequality in Theorem B.10 yields:

$$\delta_t \leq \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (2 + \mathcal{A}(t + i + 4)) \sigma_{t+i}$$

□

The bounds derived in Lemmas B.9 and B.11 depend on the definition of four successions:

- n_t , m_t , and l_t : the successions that define how the number of samples for each sampling operation varies over time.
- σ_t : the succession defining how the confidence interval of each sampling operation varies over time.

Unfortunately, due to the cumulative nature of the belief-state error, the lookahead error grows exponentially as time progresses (it grows with $\mathcal{S}^t$, and the state size $\mathcal{S}$ of the POMDP can be extremely large by itself!). This effect could be counteracted by an appropriate choice for both l_t and σ_t , but this work does not try to answer this question. Instead, a much simpler case is considered: both n_t, m_t, l_t , and σ_t are fixed, such that they do not change over time, as stated in Definition B.12.

Definition B.12. *The lookahead algorithm's samples n_t, m_t and l_t and the confidence interval $1 - \sigma_t$ are considered to be time-invariant:*

$$n_t = n, \quad m_t = m, \quad l_t = l, \quad \sigma_t = \sigma$$

Moreover, a further simplification is assumed to reduce the degrees of freedom in the choice of the number of samples: the contribution of each sampling operation to the whole lookahead error at time-step $t = 0$ is assumed to be the same, such that the number of samples n , m , and l can be related to each other according to Lemma B.13:

Lemma B.13. *If the contribution to the error bound in Theorem B.9 of each sampling operation is considered to be the same at time-step $t = 0$, the number of samples can be related to each other according to the following expressions:*

$$m = (\gamma\Gamma)^2 n, \quad l = \left(\frac{1}{4} \frac{\mathcal{R} + \gamma\Gamma\Omega}{\mathcal{S} - 1} \left(\frac{\mathcal{S}}{\Gamma(1 - \gamma^H)} \frac{(\gamma\mathcal{S})^H - 1}{\gamma\mathcal{S} - 1} - 1 \right) \right)^2 n$$

Proof. According to Theorem B.9 and Definition B.12, the lookahead error at time $t = 0$ has the following upper-bound:

$$\begin{aligned} \epsilon_0 \leq r_{\max} \sum_{k=0}^{H-1} \gamma^k \left(4\mathcal{H}(n, \sigma) + 4\gamma\Gamma\mathcal{H}(m, \sigma) + (\mathcal{R} + \gamma\Gamma\Omega) \mathcal{S}^k \mathcal{H}(l, \sigma) \sum_{j=0}^k \frac{1}{\mathcal{S}^j} \right) \\ + 2\gamma^H \Gamma r_{\max} = S_n + S_m + S_l + 2\gamma^H \Gamma r_{\max} \end{aligned}$$

where S_n, S_m and S_l represent the sums over the number of samples n, m and l , respectively. For their contributions to be the same, as stated in Lemma B.13, all of these sums must be equal ($S_n = S_m = S_l$).

Letting $S_n = S_m$ ¹³ yields:

$$\begin{aligned} 4r_{\max} \mathcal{H}(n, \sigma) \sum_{k=0}^{H-1} \gamma^k &= 4\gamma\Gamma r_{\max} \mathcal{H}(m, \sigma) \sum_{k=0}^{H-1} \gamma^k \\ \sqrt{n^{-1}} &= \gamma\Gamma \sqrt{m^{-1}} \\ m &= (\gamma\Gamma)^2 n \end{aligned}$$

Finally, making $S_n = S_l$ and resorting to Lemma B.22:

$$\begin{aligned} 4r_{\max} \mathcal{H}(n, \sigma) \sum_{k=0}^{H-1} \gamma^k &= r_{\max} \mathcal{H}(l, \sigma) (\mathcal{R} + \gamma\Gamma\Omega) \sum_{k=0}^{H-1} \gamma^k \mathcal{S}^k \sum_{j=0}^k \frac{1}{\mathcal{S}^j} \\ 4\sqrt{n^{-1}} \sum_{k=0}^{H-1} \gamma^k &\stackrel{(B.22)}{=} \sqrt{l^{-1}} (\mathcal{R} + \gamma\Gamma\Omega) \sum_{k=0}^{H-1} \gamma^k \frac{\mathcal{S}^{k+1} - 1}{\mathcal{S} - 1} \\ 4\sqrt{n^{-1}} \Gamma (1 - \gamma^H) &= \sqrt{l^{-1}} (\mathcal{R} + \gamma\Gamma\Omega) \frac{1}{\mathcal{S} - 1} \left(\mathcal{S} \sum_{k=0}^{H-1} (\gamma\mathcal{S})^k - \sum_{k=0}^{H-1} \gamma^k \right) \\ 4\sqrt{n^{-1}} \Gamma (1 - \gamma^H) &\stackrel{(B.22)}{=} \sqrt{l^{-1}} (\mathcal{R} + \gamma\Gamma\Omega) \frac{1}{\mathcal{S} - 1} \left(\mathcal{S} \frac{(\gamma\mathcal{S})^H - 1}{\gamma\mathcal{S} - 1} - \Gamma (1 - \gamma^H) \right) \\ l &= \left(\frac{1}{4} \frac{\mathcal{R} + \gamma\Gamma\Omega}{\mathcal{S} - 1} \left(\frac{\mathcal{S}}{\Gamma(1 - \gamma^H)} \frac{(\gamma\mathcal{S})^H - 1}{\gamma\mathcal{S} - 1} - 1 \right) \right)^2 n \end{aligned}$$

□

Given Definition B.12 and Lemma B.13, the bounds in Lemmas B.9 and B.11 can be re-written as follows:

¹³Recall that $\mathcal{H}(n, \sigma) = \sqrt{\frac{1}{2n} \log \left(\frac{2}{\sigma} \right)}$

Theorem B.14. *The algorithm's error ϵ_t has the following upper bound, with a confidence interval of $1 - \delta_t$:*

$$\begin{aligned}\epsilon_t &\leq r_{max} \sqrt{\frac{1}{2n} \log \left(\frac{2}{\sigma} \right)} (8\Gamma + 4\mathcal{S}^t) + 2\gamma\Gamma r_{max} \\ \delta_t &\leq \sigma (\mathcal{A}\Omega)^H \left(t + \mathcal{A} \left(8 + \Omega \frac{H-1}{(\mathcal{A}\Omega - 1)^2} \right) \right)\end{aligned}$$

Proof. The bound on error ϵ_t follows directly from Theorem B.9, by substituting m and l using the expressions in Lemma B.13.

As for the bound for the confidence δ_t , it follows from Lemma B.11 that:

$$\begin{aligned}\delta_t &\leq \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (2 + \mathcal{A}(t + i + 4)) \sigma_{t+i} \\ &= \sigma \sum_{i=0}^{H-1} \mathcal{A}^i \Omega^i (2 + \mathcal{A}(t + 4) + i\mathcal{A})\end{aligned}$$

By using the geometric series of Lemma B.22:

$$\begin{aligned}\delta_t &\stackrel{(B.22)}{\leq} \sigma \left(\mathcal{A} \sum_{i=0}^{H-1} i \mathcal{A}^i \Omega^i + (2 + \mathcal{A}(t + 4)) \frac{(\mathcal{A}\Omega)^H - 1}{\mathcal{A}\Omega - 1} \right) \\ &\leq \sigma \left(\mathcal{A} \sum_{i=0}^{H-1} i \mathcal{A}^i \Omega^i + (2 + \mathcal{A}(t + 4)) (\mathcal{A}\Omega)^H \right)\end{aligned}$$

Analogously applying a variation of the finite geometric series given by Lemma B.23 to the sum $\sum_{i=0}^{H-1} i \mathcal{A}^i \Omega^i$, the bound becomes:

$$\begin{aligned}\delta_t &\stackrel{(B.23)}{\leq} \sigma \left(\mathcal{A} \frac{(H-1)(\mathcal{A}\Omega)^{H+1} - H(\mathcal{A}\Omega)^H + \mathcal{A}\Omega}{(\mathcal{A}\Omega - 1)^2} + (2 + \mathcal{A}(t + 4)) (\mathcal{A}\Omega)^H \right) \\ &\leq \sigma \left(\mathcal{A} \frac{(H-1)(\mathcal{A}\Omega)^{H+1}}{(\mathcal{A}\Omega - 1)^2} + (2 + \mathcal{A}(t + 4)) (\mathcal{A}\Omega)^H \right) \\ &\leq \sigma (\mathcal{A}^2 \Omega)^H \left(\mathcal{A}\Omega \frac{(H-1)}{(\mathcal{A}\Omega - 1)^2} + \mathcal{A}(t + 4) + 2 \right) \\ &\leq \sigma (\mathcal{A}\Omega)^H \left(2 + \mathcal{A} \left(t + 4 + \mathcal{A}\Omega \frac{H-1}{(\mathcal{A}\Omega - 1)^2} \right) \right)\end{aligned}$$

□

As can be seen in Theorem B.14, both the bounds for ϵ_t and δ_t grow over time. Suppose that the designed algorithm accepts three arguments as input: the maximum error ϵ , the minimum confidence interval $1 - \delta$, and the stopping time T for the decision-making algorithm. In this scenario, both ϵ_t and δ_t will have a maximum bound for $t = T$. To ensure the error ϵ and the confidence interval $1 - \delta$, Theorem B.14 can be re-expressed to derive a lower-bound for the number of samples and an upper bound for σ (the probability

of failure for the sampling bounds):

$$\begin{aligned} n &\geq \frac{1}{2} \log \left(\frac{2}{\sigma} \right) \left(\frac{r_{\max}}{\epsilon - 2\gamma^H \Gamma r_{\max}} (8\Gamma + 4\mathcal{S}^T) \right)^2 \\ \sigma &\leq \delta (\mathcal{A}\Omega)^{-H} \left(2 + \mathcal{A} \left(T + 4 + \mathcal{A}\Omega \frac{H-1}{(\mathcal{A}\Omega - 1)^2} \right) \right)^{-1} \end{aligned} \quad (\text{B.7})$$

The above conditions, however, are not enough to ensure the error ϵ . This is because no matter how many samples one extracts for approximating the probability distributions, the number of samples can't compensate for the error of performing a finite lookahead if the chosen horizon H is too shallow¹⁴. As such, it should also be assured that the lookahead horizon H is large enough to make the desired error ϵ attainable. This condition is met when $2\gamma^H \Gamma r_{\max} < \epsilon$, such that:

$$H > \log_{\gamma} \left(\frac{\epsilon}{2\Gamma r_{\max}} \right) \quad (\text{B.8})$$

The three bounds given by Equations (B.7) and (B.8) sum up the whole sample complexity analysis of the lookahead algorithm. They are captured in Theorem B.15, the main theorem of this analysis:

Theorem B.15. *Consider a stopping time T , horizon H , and Definition B.12 and Lemma B.13 for the lookahead algorithm. A near-optimal action is guaranteed to be taken at each time step of the decision-making process with a regret of at most ϵ and confidence interval of at least $1 - \delta$ if all of the following conditions are met:*

$$\begin{aligned} n &\geq \frac{1}{2} \log \left(\frac{2}{\sigma} \right) \left(\frac{r_{\max}}{\epsilon - 2\gamma^H \Gamma r_{\max}} (8\Gamma + 4\mathcal{S}^T) \right)^2 \\ \sigma &\leq \delta (\mathcal{A}\Omega)^{-H} \left(2 + \mathcal{A} \left(T + 4 + \mathcal{A}\Omega \frac{H-1}{(\mathcal{A}\Omega - 1)^2} \right) \right)^{-1} \\ H &> \log_{\gamma} \left(\frac{\epsilon}{2\Gamma r_{\max}} \right) \end{aligned}$$

B.2. Time Complexity

To calculate the computational complexity of the lookahead algorithm, in both the classical and quantum-classical versions, one must multiply the total number of samples required by the computational complexity necessary to compute each sample. Given that the computational complexity for each sample has already been defined, the only task left is to define the total number of samples that need to be extracted.

In order to do so, it is important to recall that there are three types of sampling operations that the algorithm performs:

- *Reward sampling:* this sampling operation is performed at every observation node and requires n samples.

¹⁴Using a finite lookahead introduces an error of $2\gamma^H \Gamma r_{\max}$, as in Theorem B.14.

- *Observation sampling*: performed at every observation node and requires m samples.
- *Belief sampling*: performed at every belief node and requires l samples.

As such, the total number of samples N_s can be defined in terms of the number of belief nodes N_b and the number of observation nodes N_o as:

$$N_s = lN_b + (n + m) N_o$$

where the number of belief nodes is computed as

$$N_b = \sum_{i=0}^{H-1} (\mathcal{A}\Omega)^i = \frac{(\mathcal{A}\Omega)^H - 1}{\mathcal{A}\Omega - 1}$$

and the number of observation nodes as

$$N_o = \mathcal{A} \sum_{i=0}^{H-1} (\mathcal{A}\Omega)^i = \mathcal{A} \frac{(\mathcal{A}\Omega)^H - 1}{\mathcal{A}\Omega - 1}$$

such that the total number of samples can be re-expressed as

$$N_s = \frac{(\mathcal{A}\Omega)^H - 1}{\mathcal{A}\Omega - 1} (l + \mathcal{A}(n + m))$$

Let C_n , C_m , and C_l represent the computational complexity of extracting each of the three types of samples considered. In asymptotic terms, where $\mathcal{S}$, $\mathcal{A}$, $\mathcal{R}$, Ω , H , and T are all assumed to be very large, the computational complexity of the algorithm can be expressed as:

$$\mathcal{O}(\mathcal{A}^{H-1}\Omega^{H-1}(lC_l + \mathcal{A}(nC_n + mC_m)))$$

Recalling the relation between the number of samples n , m and l established in Lemma B.13, this complexity becomes:

$$\begin{aligned} & \mathcal{O}\left(n\mathcal{A}^{H-1}\Omega^{H-1}\left(\left(\frac{1}{4}\frac{\mathcal{R} + \gamma\Gamma\Omega}{\mathcal{S} - 1}\left(\frac{\mathcal{S}}{\Gamma(1 - \gamma^H)}\frac{(\gamma\mathcal{S})^H - 1}{\gamma\mathcal{S} - 1} - 1\right)\right)^2 C_l + \mathcal{A}(C_n + \gamma^2\Gamma^2 C_m)\right)\right) \\ &= \mathcal{O}\left(n\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A}(C_n + C_m) + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 C_l\right)\right) \end{aligned} \quad (\text{B.9})$$

To substitute the computational complexities C_n , C_m , and C_l , a distinction has to be made for the classical and quantum-classical algorithms, since they use different inference algorithms to extract probability distributions.

In the classical case, direct sampling is used for performing inference on the rewards and observations, while belief updating uses rejection sampling. The computational complexities are of order $\mathcal{O}(NM)$ for direct sampling and $\mathcal{O}(NMP(e)^{-1})$ for rejection sampling.

What does the evidence stand for in the case of rejection sampling? As this operation is performed at every observation node in the lookahead tree, to account for all these sampling operations, it is the mean inverse probability of evidence for belief updating. Since belief updating only uses observation RVs

as evidence, then it is given by $c_l = \sum_{o_{t+1}} P(o_{t+1}|\mathbf{b}_t, a_t)^{-1}$, where the sum is over all observations in the lookahead tree, conditioned on the previous belief state $\mathbf{b}_t$ and previous action a_t .

For the quantum-classical case, direct sampling is still used for reward and observation sampling, having the same complexity as the classical case. Quantum rejection sampling is, however, applied for belief updating. Therefore, in the quantum-classical case, the complexity for C_l is of the order $\mathcal{O}(N2^M q_l)$, where $q_l = P(e)^{-\frac{1}{2}} = \sum_{o_{t+1}} P(o_{t+1}, \mathbf{b}_t, a_t)^{-\frac{1}{2}}$. In this scenario, c_l is no longer adequate to represent $P(e)^{-\frac{1}{2}}$, because the latter is an average of the inverse square rooted probabilities $P(o_{t+1}|\mathbf{b}_t, a_t)^{-\frac{1}{2}}$ and not of the inverse probabilities $P(o_{t+1}|\mathbf{b}_t, a_t)^{-1}$.

It is also important to mention that the complexity of the algorithm can be written both in terms of the number of samples n or in terms of the precision ϵ and confidence interval $1 - \delta$, since they can be related according to Theorem B.15:

$$\mathcal{O}(n) \leq \mathcal{O}\left(\left(\frac{\mathcal{S}^T}{\epsilon}\right)^2 \log\left(\frac{1}{\sigma}\right)\right) \leq \mathcal{O}\left(\left(\frac{\mathcal{S}^T}{\epsilon}\right)^2 \log\left(\left(T + \frac{H}{\mathcal{A}\Omega}\right) \frac{\mathcal{A}^{H+1}\Omega^H}{\delta}\right)\right) \quad (\text{B.10})$$

In this context, Theorem B.16 characterizes the computational complexity of the classical lookahead algorithm:

Theorem B.16. *The computational complexity of the classical lookahead algorithm is given by any of the following expressions:*

$$\begin{aligned} & \mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 c_l\right)\right) \\ & \mathcal{O}\left(NM\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 c_l\right)\left(\frac{\mathcal{S}^T}{\epsilon}\right)^2 \log\left(\left(T + \frac{H}{\mathcal{A}\Omega}\right) \frac{\mathcal{A}^{H+1}\Omega^H}{\delta}\right)\right) \end{aligned}$$

Proof. This result is directly attained with Equations (B.9) and (B.10) and using $C_n = C_m = \mathcal{O}(NM)$ and $C_l = \mathcal{O}(NM c_l)$. $\square$

Theorem B.17 is analogous to the previous theorem, but characterizes the computational complexity of the quantum-classical lookahead algorithm:

Theorem B.17. *The computational complexity of the quantum-classical lookahead algorithm is given by any of the following expressions, whenever $2^M \approx M$:*

$$\begin{aligned} & \mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 q_l\right)\right) \\ & \mathcal{O}\left(NM\mathcal{A}^{H-1}\Omega^{H-1}\left(\mathcal{A} + ((\mathcal{R} + \Omega)\mathcal{S}^{H-1})^2 q_l\right)\left(\frac{\mathcal{S}^T}{\epsilon}\right)^2 \log\left(\left(T + \frac{H}{\mathcal{A}\Omega}\right) \frac{\mathcal{A}^{H+1}\Omega^H}{\delta}\right)\right) \end{aligned}$$

Proof. This result is directly attained with Equations (B.9) and (B.10) and using $C_n = C_m = \mathcal{O}(NM)$ and $C_l = \mathcal{O}(N2^M q_l)$. $\square$

As can be seen from Theorems B.16 and B.17, a partial speedup is obtained in the quantum case for the belief-state sampling operations, while the computational complexity of the rest of the algorithm is similar across both classical and quantum algorithms. This is so whenever the dynamic decision networks at hand are sparse enough (such that $2^M \approx M$), just as in the case of quantum rejection sampling. However, for some specific situations, the portion of the algorithm that uses a quantum subroutine dominates the complexity of the algorithm so much that the contribution of the classical subroutines to the computational complexity of the algorithm is negligible. In these scenarios, detailed in Corollaries B.18 and B.19, the quantum-classical lookahead has the greatest possible speedup over its classical counterpart:

Corollary B.18. *Under the assumption that $\frac{1}{q_l} \ll \frac{((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}}$, the expressions for the computational complexity of the classical lookahead algorithm of Theorem B.16 can be re-written as:*

$$\mathcal{O}\left(nNM\mathcal{A}^{H-1}\Omega^{H-1}((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2 c_l\right) \\ \mathcal{O}\left(NM\mathcal{A}^{H-1}\Omega^{H-1}\left((\mathcal{R}+\Omega)\frac{\mathcal{S}^{T+H-1}}{\epsilon}\right)^2 c_l \log\left(\left(T+\frac{H}{\mathcal{A}\Omega}\right)\frac{\mathcal{A}^{H+1}\Omega^H}{\delta}\right)\right)$$

Proof. Given that $q_l \leq c_l$, it follows that $\frac{1}{c_l} \leq \frac{1}{q_l}$, and therefore $\frac{1}{q_l} \ll \frac{((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}} \Rightarrow \frac{c_n}{c_l} \ll \frac{((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}}$. Thus, the result of this corollary follows directly from Theorem B.16. $\square$

Corollary B.19. *Under the assumption that $\frac{1}{q_l} \ll \frac{((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}}$, the expressions for the computational complexity of the quantum-classical lookahead algorithm of Theorem B.17 can be re-written as:*

$$\mathcal{O}\left(nN2^M\mathcal{A}^{H-1}\Omega^{H-1}((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2 q_l\right) \\ \mathcal{O}\left(N2^M\mathcal{A}^{H-1}\Omega^{H-1}\left((\mathcal{R}+\Omega)\frac{\mathcal{S}^{T+H-1}}{\epsilon}\right)^2 q_l \log\left(\left(T+\frac{H}{\mathcal{A}\Omega}\right)\frac{\mathcal{A}^{H+1}\Omega^H}{\delta}\right)\right)$$

Corollaries B.18 and B.19 compare the classical and quantum algorithm's complexity in the best-case scenario for the quantum algorithm, where $\frac{c_n}{q_l} \ll \frac{((\mathcal{R}+\Omega)\mathcal{S}^{H-1})^2}{\mathcal{A}}$. It remains to be answered as to what RL problems this assumption can be applied to in practice, as this work does not address this question. Nonetheless, it gives a benchmark as to what speedup one can expect from the quantum algorithm in a best-case scenario.

What speedups can therefore be expected? Although one might initially think that using quantum rejection sampling would provide a quadratic speedup, this is not always the case. Dividing the classical complexity by the quantum complexity (and assuming that M and 2^M have about the same order of magnitude), this fraction reduces to $\frac{c_l}{q_l} = \frac{\sum_x P^{-1}(x)}{\sum_x P^{-\frac{1}{2}}(x)}$, where the sum over x represents the sum over the evidence. From inequality $\sqrt{\sum_x a_x} \leq \sum_x \sqrt{a_x} \leq \sum_x a_x$, it is easy to derive that:

$$1 \leq \frac{c_l}{q_l} \leq q_l \quad (\text{B.11})$$

The inequality in Equation (B.11) entails that the complexity of the quantum algorithm relative to the classical one can range between no speedup to a quadratic one, but it is most likely to fall somewhere in between these two extremes. This comes from the notion that, although using quantum rejection sampling yields a quadratic advantage over classical rejection sampling, the same cannot be said about using it sequentially, due to the mathematical property that the sum of square roots can be (and usually is) larger than the square root of the sum.

B.3. Auxiliary lemmas

To derive an upper bound on the lookahead algorithm's error of Definition B.1, some lemmas are of use. One example is Lemma B.20, an inequality that relates the difference of the maxima of two vectors and the maximum of their difference:

Lemma B.20. *Let $\mathbf{U}, \mathbf{V} \in \mathcal{W}$ be vectors in some vector space $\mathcal{W}$. Then:*

$$\|\mathbf{U}\|_\infty - \|\mathbf{V}\|_\infty \leq \|\mathbf{U} - \mathbf{V}\|_\infty$$

Proof. Let U_i and V_i represent entries of $\mathbf{U}$ and $\mathbf{V}$, respectively. Then:

$$\begin{aligned} U_i &\leq |U_i - V_i| + V_i \\ \max_i U_i &\leq \max_i (|U_i - V_i| + V_i) \\ \max_i U_i &\leq \max_i |U_i - V_i| + \max_i V_i \\ \max_i U_i - \max_i V_i &\leq \max_i |U_i - V_i| \\ \|\mathbf{U}\|_\infty - \|\mathbf{V}\|_\infty &\leq \|\mathbf{U} - \mathbf{V}\|_\infty \end{aligned}$$

□

Another result that is used repeatedly throughout the proofs is a generalization of the triangle's inequality, stated in Lemma B.21:

Lemma B.21. *Let $\mathbf{U}, \mathbf{V} \in \mathcal{W}$ be vectors in an Euclidean space $\mathcal{W}$. Then:*

$$\|\mathbf{U} + \mathbf{V}\| \leq \|\mathbf{U}\| + \|\mathbf{V}\|$$

Proof.

$$\|\mathbf{U} + \mathbf{V}\|^2 = \|\mathbf{U}\|^2 + \|\mathbf{V}\|^2 + 2\mathbf{U}^\top \mathbf{V}$$

Using the Cauchy-Schwarz inequality, stating that $|\mathbf{U}^\top \mathbf{V}| \leq \|\mathbf{U}\| \|\mathbf{V}\|$, we get:

$$\begin{aligned} \|\mathbf{U} + \mathbf{V}\| &\leq \sqrt{\|\mathbf{U}\|^2 + \|\mathbf{V}\|^2 + 2\|\mathbf{U}\| \|\mathbf{V}\|} \\ \|\mathbf{U} + \mathbf{V}\| &\leq \|\mathbf{U}\| + \|\mathbf{V}\| \end{aligned}$$

□

Lemma B.22 simplifies a finite geometric series into a fraction:

Lemma B.22. *The finite geometric series can be simplified according to the following expression Herman & Strang (2016):*

$$\sum_{i=0}^N z^i = \frac{z^N - 1}{z - 1}$$

Finally, a variation of the finite geometric series is also simplified in Lemma B.23:

Lemma B.23. *A variation of the finite geometric series of Lemma B.22 can be simplified according to the following expression Herman & Strang (2016):*

$$\sum_{i=0}^N iz^i = \frac{(N-1)z^{N+1} - Nz^N + z}{(z-1)^2}$$